

03 - Introducción a Ciencia de Datos (Introducción + Métricas de Clasificación + Clasificación con SGD)

1) Introducción a la Ciencia de Datos

1.1 Modelos y tipos de variables

- Modelos: según el objetivo, se usan modelos de clasificación (predecir categorías), regresión (valores continuos), agrupamiento/clustering (descubrir grupos sin etiquetas), transformación, asociación, etc.
- Variables: independientes (entradas) y dependientes (salidas/categorías).
- Tipos de variables independientes:
 - Cualitativas: nominales (categorías sin orden) y ordinales (con orden: p. ej., poco/medio/mucho).
 - Cuantitativas: discretas (conteos) y continuas (mediciones).
- Tipos de problemas según la variable dependiente:
 - Si es cualitativa → clasificación.
 - Si es cuantitativa → regresión.
 - Si no hay variable dependiente → agrupamiento (clustering).

1.2 Estadística básica: varianza, covarianza y correlación

- Varianza muestral (de $x_1, \dots, x_n$, media $\bar{x}$):
$$s^2 = (1/(n-1)) \cdot \sum_i (x_i - \bar{x})^2$$
(Se divide por $n-1$ al estimar con muestra).
- Desvío estándar muestral: $s = \sqrt{s^2}$.
- Covarianza muestral entre X e Y (medias $\bar{x}, \bar{y}$):
$$\text{cov}(X,Y) = (1/(n-1)) \cdot \sum_i (x_i - \bar{x}) \cdot (y_i - \bar{y})$$
Indica variación conjunta; signo positivo/negativo sugiere relación lineal creciente/decreciente.
- Correlación de Pearson r (adimensional, en $[-1,1]$):
$$r = \text{cov}(X,Y) / (s_x \cdot s_y)$$
 $r \approx 0$: sin correlación lineal; $r = \pm 1$: relación lineal perfecta. Correlación no implica causalidad.

1.3 Regresión lineal

- Objetivo: predecir una variable continua y a partir de entradas. Para una sola variable x:
$$\hat{y} = a + b \cdot x$$
- Estimadores por mínimos cuadrados (con n observaciones):
$$b = [n \cdot \sum(x_i y_i) - (\sum x_i)(\sum y_i)] / [n \cdot \sum(x_i^2) - (\sum x_i)^2]$$
$$a = (\sum y_i - b \cdot \sum x_i) / n$$
- Forma matricial (Ecuación normal) para múltiples variables: con $X \in \mathbb{R}^{m \times (p+1)}$ (incluye columna de 1s) e $y \in \mathbb{R}^m$:
$$\hat{\theta} = (X^T X)^{-1} X^T y$$
Complejidad de invertir $X^T X \approx O(p^2 \dots p^3)$.
- Métricas de error en regresión (con predicciones $h(x_i)$ y observaciones y_i):
 - Error cuadrático medio (MSE): $\text{MSE} = (1/m) \cdot \sum_i (y_i - h(x_i))^2$
 - Raíz del error cuadrático medio (RMSE): $\text{RMSE} = \sqrt{\text{MSE}}$
 - Error absoluto medio (MAE): $\text{MAE} = (1/m) \cdot \sum_i |y_i - h(x_i)|$

1.4 Clasificación y Regresión Logística

- Clasificación: asignar un ejemplo x a una categoría $c \in C$ conocida.
- Regresión logística (binaria): modela $P(y=1|x)$ con la función sigmoide $\sigma(z)$:
 $\sigma(z) = 1 / (1 + e^{-z})$, con $z = w^T x + b$
Regla de decisión con umbral t : predecir 1 si $\sigma(z) \geq t$; de lo contrario 0.

1.5 Clustering (K-Means) y elección de K

- K-Means (resumen): elegir K ; inicializar centroides; asignar cada punto al centroide más cercano; recalcular centroides como promedio de puntos asignados; iterar hasta convergencia.
- Criterios para K :
 - Regla del codo: observar la inercia/"within-cluster sum of squares (WCSS)" y buscar el "codo".
 - Silhouette: para cada punto i , calcular $a(i)$ (distancia promedio a su propio cluster) y $b(i)$ (distancia promedio al cluster más cercano distinto). Coeficiente:
 $s(i) = [b(i) - a(i)] / \max\{a(i), b(i)\} \in [-1, 1]$
Valores cercanos a 1 indican buena asignación; cercanos a -1 sugieren mala asignación.
- Estadística de Hopkins H (tendencia al agrupamiento):
 1. Para datos reales D tomar n puntos y distancias al vecino más cercano: $x_1, \dots, x_n$.
 2. Generar random D uniforme y medir distancias al vecino real más cercano: $y_1, \dots, y_n$.
 3. Definir $H = (\sum_i y_i) / (\sum_i x_i + \sum_i y_i)$.
Interpretación: $H \approx 0.5$ sugiere aleatoriedad; $H \gg 0.5$ sugiere estructura de clusters (p. ej., $H > 0.75$ indica fuerte tendencia).

1.5 Entrenamiento/validación y sobreajuste

- Separar conjuntos de entrenamiento y prueba; opcionalmente validación. Datos balanceados mejoran evaluación.
- Overfitting: modelo se ajusta al ruido del train y generaliza mal; se mitiga con regularización, más datos, validación adecuada, etc.

2) Métricas de Clasificación

Definiciones base (binario, clases positiva/negativa):

- TP (True Positives): positivos correctamente clasificados.
- TN (True Negatives): negativos correctamente clasificados.
- FP (False Positives): negativos mal clasificados como positivos.
- FN (False Negatives): positivos mal clasificados como negativos.

Matriz de confusión (binaria):

	Predicho +	Predicho -
Real +	TP	FN
Real -	FP	TN

En multiclase se generaliza a matriz $n \times n$; la diagonal contiene aciertos.

$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$	La <i>precision</i> mide la exactitud de las predicciones positivas del modelo. Se define como la proporción de positivos correctos sobre el total de elementos clasificados como positivos. Una precision alta significa que cuando el modelo dice “positivo”, casi siempre acierta.
$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$	El <i>recall</i> mide la capacidad del modelo para encontrar todos los casos positivos reales. Se define como la proporción de positivos correctamente identificados entre todos los positivos existentes en el conjunto de datos. Un recall alto significa que casi no se dejan escapar casos positivos.
$\begin{aligned} F\beta \text{ Score} &= \frac{1 + \beta^2}{\frac{1}{\text{Precision}} + \frac{\beta^2}{\text{Recall}}} \\ &= \frac{(1 + \beta^2) \times \text{Precision} \times \text{Recall}}{(\beta^2 \times \text{Precision}) + \text{Recall}} \\ F1 \text{ Score} &= \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} \\ &= \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \end{aligned}$	El <i>F-score</i> es la media armónica entre <i>precision</i> y <i>recall</i>. Se usa para balancear ambas métricas, especialmente cuando se busca un equilibrio entre no perder positivos y no clasificar erróneamente demasiados negativos como positivos. Es muy útil cuando las clases están desbalanceadas, ya que ni la precision ni el recall por separado son suficientes para evaluar bien el modelo.
$\text{Accuracy} = \frac{(\text{TP} + \text{TN})}{(\text{TP} + \text{FP} + \text{TN} + \text{FN})}$	La <i>accuracy</i> mide el porcentaje de predicciones correctas (positivas y negativas) respecto al total. Es útil cuando las clases están balanceadas, pero puede ser engañosa si los datos son muy desbalanceados (por ejemplo, cuando hay muchos más negativos que positivos).

- Especificidad (TNR): $\text{TNR} = \text{TN} / (\text{TN} + \text{FP})$
- Tasa de falsos positivos (FPR): $\text{FPR} = \text{FP} / (\text{FP} + \text{TN}) = 1 - \text{TNR}$
- Medida F1 (armónica de P y R): $\text{F1} = 2 \cdot (\text{P} \cdot \text{R}) / (\text{P} + \text{R})$

Trade-off Precisión–Recall: al variar umbral t , $P \uparrow$ suele implicar $R \downarrow$ y viceversa; elegir umbral según objetivo del problema.

Curva ROC: gráfica TPR (=Recall) vs. FPR para todos los umbrales. Área bajo la curva (AUC):

- AUC = 1: clasificador perfecto.
- AUC $\approx$ 0.5: azar.

3) Descenso por Gradiente (GD) y SGD

- Descenso por Gradiente (GD): minimiza una función de pérdida $L(\theta)$ actualizando parámetros en la dirección opuesta al gradiente:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} L(\theta_t)$$

donde η es el learning rate ($0 < \eta \leq 1$ típico).

Ejemplo (intersección y con pendiente fija m): pérdida MSE en 1D con residuo $r_i = y_i - (y + m \cdot x_i)$:

$$L(y) = (1/m) \cdot \sum_i r_i^2 \Rightarrow \partial L / \partial y = (-2/m) \cdot \sum_i r_i$$

Actualización: $y_{t+1} = y_t - \eta \cdot (\partial L / \partial y)$.
- SGD (Stochastic Gradient Descent): en lugar de usar todo el conjunto por paso (batch), usa un ejemplo (o minibatch) al azar por iteración:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} L_i(\theta_t)$$

Reduce costo por paso y puede escapar de mínimos locales; sensible a η , tamaño de minibatch y número de épocas.

Umbral y decision_function: en clasificadores lineales (p. ej., SGDClassifier) se puede usar un puntaje $s(x)$ y fijar un umbral t para decidir la clase, lo que permite ajustar el equilibrio Precisión/Recall.

Ejercicio 11

Se entrenó un modelo de clasificación para detectar la especie de una flor : clase 0, clase 1 y clase 2. Luego se evaluó el modelo en los datos de test y se obtuvo la siguiente matriz de confusión:

	0	1	2
0	13	1	1
1	0	15	3
2	4	1	13
	0	1	2

Se pide:

- Calcular la métrica Precision para la clase 0: _____
- Calcular la métrica Recall para la clase 1: _____
- Calcular el accuracy del modelo: _____

Precisión para la clase 0: $TP / (TP + FP) = 13 / (13 + 4) = 13/17$

Recall para la clase 1: $TP / (TP + FN) = 15 / (15 + 3) = 15/18$

Accuracy del modelo: (diagonal) / (observaciones) = $(13 + 15 + 13) / (13 + 1 + 1 + 0 + 15 + 3 + 4 + 1 + 13) = 41/51$

Ejercicio 12

Para un proyecto de ciencia de datos, se entrena un modelo predictivo y se evalúa su performance con la métrica F1-Score. Se obtienen los siguientes resultados:

-F1-Score en entrenamiento: 0.75

-F1-Score en test: 0.31

Indique cuáles si las siguientes afirmaciones son verdaderas o falsas (V/F):

- El modelo generaliza muy bien para datos nuevos. _____
- El modelo podría estar sobreajustando a los datos de entrenamiento (overfitting). _____
- Hay un problema en los datos de test, se debería evaluar en otro conjunto. _____
- El modelo podría estar subajustando a los datos de entrenamiento (underfitting). _____

a) Falso. Con $F1=0.31$ en test (muy por debajo de 0.75 en train), el modelo no está generalizando bien a datos nuevos.

b) Verdadero. Alto rendimiento en entrenamiento y mucho peor en test es el patrón típico de sobreajuste (overfitting).

c) Falso (no se infiere). No hay evidencia de que el set de test esté "mal"; lo más probable es el sobreajuste del modelo. Podrías validar con k-fold CV, revisar el split y el preprocesamiento, pero no es correcto asumir un problema en el test por estos valores.

d) Falso. Underfitting se ve cuando el desempeño es bajo tanto en train como en test. Aquí el F1 en train (0.75) es razonable; el problema es la caída en test.

Ejercicio 13

Explique brevemente las principales diferencias entre métodos de regresión y de clasificación.

Qué predicen

- Regresión: métodos que estiman un valor continuo (ej.: temperatura, precio).
- Clasificación: métodos que asignan una categoría dentro de un conjunto finito y conocido de clases.

Cómo modelan/salen

- Regresión: producen números reales; p.ej. regresión lineal ajusta una recta (mínimos cuadrados / ecuación normal o descenso por gradiente).
- Clasificación: producen una etiqueta; a menudo un método como regresión logística entrega probabilidades 0-1 y luego se aplica un umbral para decidir la clase.

Función/criterio de ajuste típico

- Regresión: minimizar errores sobre valores continuos, como MSE/RMSE o MAE.
- Clasificación: optimizar el desempeño sobre clases; se analiza con matriz de confusión y métricas derivadas.

Métricas de evaluación típicas

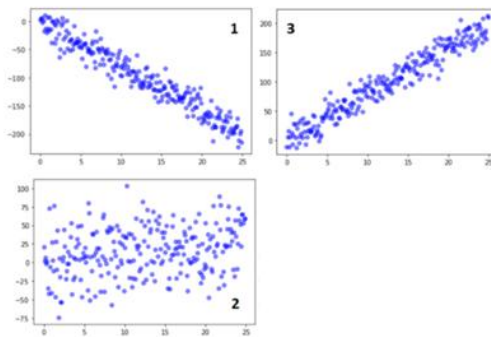
- Regresión: MAE, MSE, RMSE (error medio absoluto/cuadrático/raíz).
- Clasificación: Precision, Recall, F1 (a partir de VP, FP, FN, VN).

Ejemplos de métodos

- Regresión: Regresión lineal (mínimos cuadrados, ecuación normal, gradiente).
- Clasificación: Regresión logística (separar en clases y estimar probabilidad).

Ejercicio 2

Dados los siguientes **coeficientes de correlación de pearson**, para dos variables X e Y, realizar el gráfico de dispersión corresponde cada uno: a) $r = 0.80$ b) $r = -0.86$ c) $r = 0.35$
¿Qué puede mencionar de la relación de las variables en cada caso?



- Gráfico 1 $\rightarrow r = -0.86$ (opción b)

Se ve una tendencia lineal negativa fuerte: a medida que X aumenta, Y disminuye de forma casi lineal; la nube está “aplastada” alrededor de una recta descendente, lo que cuadra con un $|r|$ alto y signo negativo. Un r cercano a -1 indica correlación lineal negativa casi perfecta; lejos de 0 significa relación fuerte.

Comentario: aunque la asociación es clara, correlación no implica causalidad; podría haber variables no medidas explicando el patrón.

- Gráfico 2 → $r = +0.35$ (opción c)

La nube se ve dispersa, con apenas una inclinación ascendente suave: eso es una correlación positiva débil. Un r cercano a 0 indica poca relación lineal; aquí 0.35 sugiere asociación lineal modesta y bastante ruido.

Comentario: con r bajo, el poder predictivo lineal es limitado; podría haber relación no lineal o simplemente una asociación tenue. El scatter es la herramienta para detectar si hay o no tendencia visible.

- Gráfico 3 → $r = +0.80$ (opción a)

Presenta una tendencia lineal positiva fuerte: a mayor X, mayor Y, con puntos bastante alineados en sentido ascendente. Un r alto y positivo (≈ 0.8) describe justamente ese comportamiento.

Comentario: la forma de “nube alargada” alrededor de una recta ascendente es típica de una correlación positiva; el diagrama de dispersión permite visualizar esa tendencia.

Ejercicio 5

Se entrenaron dos **modelos de clasificación** para detectar personas sanas (clase 0) y personas enfermas (clase 1). Luego se evaluaron los modelos en los datos de test y se obtuvieron las siguientes **matrices de confusión**:

	Modelo A		Modelo B	
True \ Predicted	0	1	0	1
0	13	12	15	10
1	16	22	11	27

Calcular para la Clase 1:

$$F1 \text{ Score} = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

a) F1 Score Modelo A

$$\text{Recall Clase 1: } 22 / 22 + 16 = 11/19$$

$$\text{Precision Clase 1: } 22 / 22 + 12 = 11/17$$

b) F1 Score Modelo B

$$\text{Recall Clase 1: } 27 / 27 + 11 = 27 / 38$$

$$\text{Precision Clase 1: } 27 / 27 + 10 = 27 / 37$$

c) ¿Qué modelo elegiría en términos de Accuracy? ¿Por qué?

$$\text{Accuracy A: } \text{TN} + \text{TP} / \text{TN} + \text{TP} + \text{FP} + \text{FN}$$

$$\text{Accuracy A: } 13 + 22 / 13 + 22 + 16 + 12$$

$$\text{Accuracy A: } 35 / 63$$

$$\text{Accuracy B: } \text{TN} + \text{TP} / \text{TN} + \text{TP} + \text{FP} + \text{FN}$$

$$\text{Accuracy B: } 15 + 27 / 15 + 27 + 11 + 10$$

$$\text{Accuracy B: } 42 / 63$$

El modelo B tiene mayor accuracy

$$\text{Precisión A: } \text{TP} / (\text{TP} + \text{FP}) = 22 / (22 + 12) = 22 / 34$$

$$\text{Recall A: } \text{TP} / (\text{TP} + \text{FN}) = 22 / (22 + 16) = 22 / 38$$

$$\text{Precision B: } 27 / (27 + 10) = 27 / 37$$

$$\text{Recall B: } 27 / (27 + 11) = 27 / 38$$

$$\text{a) } (2 * 22/34 * 22/38) / (22/34 + 22/38) = 0,6111$$

$$\text{b) } (2 * 27/37 * 27/38) / (27/37 + 27/38) = 0,72$$

$$\text{c) Accuracy A: } 35/63 \quad \text{Accuracy B: } 42/63$$

El B tiene mayor accuracy así que lo elegimos por eso

Ejercicio 7

Mencione cuál es la diferencia entre las métricas **Precision y Recall** y cómo se calculan las mismas.

Precision: $\text{TP} / (\text{TP} + \text{FP})$

¿Cuántas veces lo que mi modelo dice es realmente cierto?

Precision se centra en lo que el modelo dice y luego lo compara con la realidad.

Recall: $\text{TP} / (\text{TP} + \text{FN})$

¿Cuántas veces mi modelo es capaz de identificar la verdad?

Recall parte de la realidad, y después evalúa que tan bueno es el modelo para reconocerla.

- a. ¿En qué tipo de problemas sería conveniente maximizar la métrica Precision y en cuáles Recall?. Ejemplifique.

Precisión cuando el costo de los falsos positivos sea alto. Por ejemplo, en el diagnóstico médico, diagnosticar erróneamente una enfermedad a una persona sana puede generar tratamientos y gastos innecesarios.

Recall cuando el costo de los falsos negativos sea alto: por ejemplo, en la detección de fraudes, no detectar una transacción fraudulenta puede provocar pérdidas financieras importantes.

- b. ¿Son esas métricas adecuadas para resolver un problema de regresión? En caso de responder negativamente, explique qué métricas usaría.

No, no son las métricas adecuadas. Para evaluar el rendimiento de los modelos en problemas de regresión se mide el error, las métricas que se pueden utilizar son MAE, MSE, RMSE etc.

Diferencia entre Precision y Recall y cómo se calculan

- Precision (Precisión) mide “qué tan limpios” son los positivos que predijo el modelo: de todo lo que predije como Positivo, qué proporción fue realmente Positivo. Se calcula como $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$, usando la matriz de confusión.
- Recall (Exhaustividad/Sensibilidad) mide “cuánto recupero” de los positivos reales: de todos los Positivos verdaderos que había, cuántos encontré. Se calcula como $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$.
- En la práctica, al mover el umbral de decisión se intercambia precisión por recall (sube una y suele bajar la otra).

a) ¿Cuándo conviene maximizar Precision y cuándo Recall? Ejemplos

- Maximizar Precision cuando un falso positivo es costoso y preferimos “disparar menos, pero con acierto”.
Ejemplos:
 - Filtro de spam “agresivo” para no mandar mails legítimos a spam: queremos que lo que va a spam sea casi seguro spam (bajar FP).
 - En el ejemplo del curso “apuesto solo si estoy muy seguro de que Argentina gana”: el objetivo pide alta precisión (pocas apuestas perdidas = pocos FP).
- Maximizar Recall cuando un falso negativo es costoso y preferimos “no perdernos ningún positivo”, aunque vengan más falsos positivos.
Ejemplos:
 - Screening médico (detección temprana): es peor no detectar un enfermo (FN). Buscamos alto recall.
 - Siguiendo el ejemplo del curso: “quiero celebrar cada juego que se gana” $\Rightarrow$ alto recall (no omitir victorias = pocos FN).

Regla mental: Precision controla la pureza de los positivos predichos (penaliza FP); Recall controla la cobertura de positivos reales (penaliza FN).

b) ¿Sirven para regresión? Si no, ¿qué métricas usaría?

No. Precision y Recall son métricas de clasificación basadas en la matriz de confusión (VP, FP, FN, VN). Para regresión (variable objetivo continua) se usan métricas de error como:

- MAE (Error Medio Absoluto),
- MSE (Error Cuadrático Medio),
- RMSE (Raíz del Error Cuadrático Medio).

Estas aparecen en la sección de métricas de regresión del material.

Ejercicio 2

Determinar si las siguientes afirmaciones son verdaderas o falsas:

- El coeficiente de correlación de pearson mide cuán relacionadas están dos variables sea o no de forma lineal. _____
- El método de regresión logística se utiliza para resolver problemas de clasificación binaria. _____
- La regresión lineal simple consiste en predecir un valor numérico en rango continuo en base a más de una variable predictora. _____

“Pearson mide cuán relacionadas están dos variables sea o no de forma lineal.” $\rightarrow$ FALSO.

El coeficiente de Pearson cuantifica solo la relación lineal; puede fallar si la relación es no lineal.

“La regresión logística se utiliza para resolver problemas de clasificación binaria.” → VERDADERO.
Modela la probabilidad de pertenecer a una de dos clases (y luego se umbraliza).

“La regresión lineal simple consiste en predecir un valor continuo en base a más de una variable predictora.” → FALSO.

Simple implica una sola variable explicativa; “más de una” es regresión lineal múltiple.

Ejercicio 4

Se entrenó un modelo de clasificación (MODELO A) para detectar personas enfermas (clase 1) y sanas (clase 0). Luego se evaluó el modelo en los datos de test y se obtuvo la siguiente matriz de confusión:

True \ Predicted	0	1
0	135	36
1	38	111

Luego se entrenó un segundo modelo de clasificación (MODELO B), se evaluó sobre los mismos datos de test y se obtuvieron los siguientes resultados:

True Positive: 49
True Negative: 130
False Positive: 41
False Negative: 100

¿Qué modelo (A o B) elegiría en términos de Accuracy?: _____

- Accuracy Modelo A _____
- Accuracy Modelo B _____

Accuracy A: $(135+111) / (135+111+38+36) = 0,768$

Accuracy B: $(49+130) / (49+130+41+100) = 0,559$

Elijo el Modelo A, porque $0,768 > 0,559$.

Recordatorio de teoría: $Accuracy = (VP + VN) / (VP + VN + FP + FN)$. Como ambos modelos se evaluaron sobre el mismo test (mismo total = 320), comparar accuracies es directo. Si la clase positiva fuera muy rara, convendría mirar también Precision/Recall/F1; pero con lo pedido (“en términos de Accuracy”), gana A.

Ejercicio 1

Explique brevemente los conceptos de **Overfitting** y **Underfitting**. Mencione cómo puede detectarse cada una de estas situaciones y qué se podría hacer para evitarlas.

*El **overfitting** es el sobreajuste del modelo a los datos de entrenamiento, el modelo aprende incluso el ruido en los datos de entrada. También hay que analizar la complejidad del modelo. Se puede detectar cuando un modelo performa muy bien en entrenamiento y no tiene poder de generalización, es decir, tiene baja performance sobre datos nuevos. Los modelos que incurrn en overfitting suelen tener alta varianza, lo que implica que son extremadamente sensibles a pequeñas alteraciones en los datos. Para evitarlo se pueden usar técnicas de regularización en los modelos, usar validación cruzada, etc.*

*El **underfitting** es cuando el modelo no logra aprender sobre los datos de entrenamiento. Un modelo con underfitting es aquel en donde los errores tanto de entrenamiento como de validación son similares y relativamente altos. Los modelos víctimas del underfitting tienen un bias alto. También hay que analizar la complejidad del modelo, ya que para evitarlo se puede usar modelos más complejos, recolectar más datos si son escasos, reducir la regularización, etc.*

Ejercicio 2

Determinar cuáles de las siguientes afirmaciones son Verdaderas o Falsas

- La varianza de un estimador es cuánto varía la predicción según los datos que utilicemos para testear. F
- Un bias muy alto significa que el modelo no se ha ajustado lo suficiente a los datos de entrenamiento. V
- Un modelo con varianza baja indica que pequeños cambios en los datos de entrenamiento producen cambios pequeños en la estimación. V
- Un bias muy alto suele ser underfitting por lo tanto se produce un error alto sólo en las muestras de test. F

“La varianza de un estimador es cuánto varía la predicción según los datos que utilicemos para testear.” → FALSO
La varianza (en el sesgo-varianza) habla de cuánta sensibilidad a cambios en el conjunto de entrenamiento tiene el modelo. Alta varianza = el modelo cambia mucho si cambiás la muestra de train; baja varianza = es estable. No depende del set de test.

“Un bias muy alto significa que el modelo no se ha ajustado lo suficiente a los datos de entrenamiento.” → VERDADERO

Alto sesgo = underfitting: el modelo es demasiado simple o está demasiado regularizado y no captura los patrones ni siquiera en entrenamiento.

“Un modelo con varianza baja indica que pequeños cambios en los datos de entrenamiento producen cambios pequeños en la estimación.” → VERDADERO

Eso es, baja varianza = estabilidad frente a perturbaciones del set de entrenamiento.

“Un bias muy alto suele ser underfitting por lo tanto se produce un error alto sólo en las muestras de test.” → FALSO

Con alto sesgo el error es alto tanto en train como en test (porque el modelo ya es incapaz de ajustarse bien a la relación verdadera). No es un problema “sólo de test”.

Ejercicio 3

Determinar si las siguientes afirmaciones sobre **K-means** son verdaderas o falsas:

- Es un algoritmo no supervisado de Clustering ya que partimos de un conjunto de datos etiquetado previamente. _____
- El parámetro K corresponde a la cantidad de clusters o grupos que se formarán. _____
- El parámetro K corresponde a la cantidad de centroides que se calcularán. _____
- Es un algoritmo supervisado de Clustering ya que partimos de un conjunto de datos etiquetado previamente. _____

a) Falso. K-means es no supervisado, pero justamente porque no parte de etiquetas; descubre grupos solo a partir de las características.

b) Verdadero. K es el número de clusters que queremos formar.

c) Verdadero. En K-means hay un centroide por cluster, así que K también es el número de centroides que se calculan/actualizan.

d) Falso. No es un algoritmo supervisado ni requiere datos etiquetados; eso sería un problema de clasificación, no de clustering.

04 – Ingeniería de Features (Análisis de valores atípicos + Procesamiento y transformación de datos)

1. Preprocesamiento y Transformación de Datos (dentro del proceso KDD)

En el proceso de KDD (Knowledge Discovery in Databases) nos concentramos en las etapas de preprocesamiento y transformación. En preprocesamiento se abordan tareas como: integración de datos (combinar múltiples bases, archivos, etc.), limpieza de datos (completar faltantes, eliminar ruido, identificar/eliminar valores atípicos y corregir incoherencias), reducción de datos (por dimensionalidad y por numerosidad) y, ya en transformación, normalizaciones, discretizaciones y feature engineering en general.

1.1 Limpieza de datos: problemas típicos y datos faltantes

Los datos pueden presentar inconsistencias (por ejemplo, unidades o formatos mezclados como “1 m / 100 cm”, o decimales escritos como “1.5 / 1,5”), errores de tipeo y casos similares. Dentro de la limpieza, un tema central son los datos faltantes.

1.1.1 Tipos de datos faltantes (Clasificación de Rubin, 1976)

- MCAR (Missing Completely At Random): la falta es completamente aleatoria; la probabilidad de estar ausente es igual para todas las observaciones y no se relaciona ni con valores observados ni con otros faltantes. Omitir estos casos no sesga las inferencias. Ejemplo: se pierde la respuesta de una encuesta.
- MAR (Missing At Random): la ausencia puede explicarse por otras variables observadas (no por la propia). No es puro azar. Ejemplo: en un ensayo clínico, algunos sujetos se retiran cuando aparecen efectos secundarios, y esa retirada se relaciona con variables observadas.
- MNAR (Missing Not At Random): la falta no es aleatoria y tiene una explicación ligada a la variable. Ejemplos: personas con ingresos altos se niegan a reportarlos; pacientes con depresión omiten responder ciertas preguntas. En estos casos conviene investigar la causa.

1.1.2 Estrategias frente a faltantes: eliminar o imputar

Primero conviene entender el motivo de ausencia para elegir el tratamiento adecuado.

- 1) Eliminar registros o variables: solo si no afecta la utilidad del dataset; se desaconseja salvo situaciones MCAR.
- 2) Imputar: reemplazar los faltantes por estimaciones. El objetivo es obtener un dataset completo para modelar. Hay enfoques univariados (cada variable por separado) y multivariados (el valor imputado depende de varias variables).

1.1.3 Técnicas de imputación (detalle)

- Sustitución de casos: un experto reemplaza el faltante por un valor no observado pero plausible según conocimiento externo.
- Media/Mediana: reemplazo simple con media o mediana. Desventajas: atenúa la varianza (valores repetidos), distorsiona la distribución y deprime correlaciones.
- Cold Deck: usa relaciones/valores de fuentes externas al dataset actual (por ejemplo, capas geográficas para imputar variables).
- Hot Deck: reemplaza con valores de registros similares (definir la métrica de similaridad; por ejemplo, K vecinos más cercanos).
- Regresión: predecir el faltante con un modelo usando variables disponibles como predictores.
- MICE (Multivariate Imputation by Chained Equations): asume MAR y realiza imputaciones iterativas por variable, usando el resto como predictores, repitiendo hasta converger (no trivial fijar criterio de convergencia).

- Imputación con KNN: estima faltantes por semejanza a vecinos cercanos; conviene emplear una métrica que tolere NaNs (por ejemplo, `nan_euclidean_distance`) y usar K impar.

1.2 Transformación de datos: Feature Engineering

El feature engineering abarca cualquier modificación de la forma de los datos para mejorar el desempeño de los modelos. Incluye normalización, discretización, transformaciones para aproximar normalidad y creación de nuevas variables.

1.2.1 Normalización: Min–Max, Z-score y decimal scaling

Se aplica a variables numéricas para llevarlas a escalas comparables y evitar que magnitudes grandes dominen.

- Min–Max (rango 0–1): escala respecto del mínimo y el rango. Útil cuando min y max reales son conocidos y no hay outliers extremos.
- Z-score: centra en la media y escala por desvío estándar; útil cuando el verdadero min/max no es claro o hay outliers que afectarían Min–Max.
- Decimal scaling: asegura que los valores normalizados queden entre -1 y 1 , escalando por el número de dígitos del valor absoluto más grande.

1.2.2 Transformaciones para lograr normalidad

Para reducir sesgos se pueden aplicar: raíz cuadrada, logaritmos, inversa de la raíz cuadrada o Box–Cox, según la distribución de la variable.

1.2.3 Discretización (Binning) y reemplazos

La discretización divide el rango de una variable continua en intervalos, reduciendo valores a un conjunto acotado de etiquetas. Criterios: igual frecuencia (mismo número de observaciones por bin), igual ancho (rangos uniformes) y cuantiles (mediana, percentiles). Luego de agrupar, se puede reemplazar por media/mediana del bin o por etiquetas/enteros.

1.2.4 Variables categóricas: One-Hot Encoding (dummies)

Muchos métodos requieren predictores numéricos; las variables categóricas se recodifican en una o más dummies (one-hot).

1.2.5 Generación de nuevas variables (imaginación)

Crear nuevas variables puede mejorar el modelo. Ejemplo: sumar fuentes de información para calcular la distancia desde cada instancia (p. ej., inmueble) al espacio/servicio más cercano.

2. Análisis de Valores Atípicos (Outliers)

Un outlier es una observación que se aleja tanto del resto que despierta sospechas de haber sido generada por un mecanismo diferente (definición clásica de D. Hawkins, 1980). Es un concepto dependiente del problema: puede deberse a error de medición, a aleatoriedad o a que la instancia pertenezca a otra “familia” (por ejemplo: una persona de 120 años; un niño de 4 años que mide 1,80 m).

2.1 ¿Por qué importan y qué hacer con ellos?

La presencia de outliers influye en resultados de análisis estadísticos clásicos. No siempre hay que eliminarlos: deben inspeccionarse con cuidado. A veces lo que interesa es detectarlos (fraudes, fallas, patologías).

2.2 Tipos de outliers: univariados y multivariados

- Univariados: atípicos en una sola variable; detectan extremos pero fallan en combinaciones más sutiles.
- Multivariados: atípicos en un espacio n-dimensional; requieren ajustar modelos que consideren la relación entre variables.

2.3 Métodos univariados: IQR, Z-score y Z-score modificado (MAD)

- IQR/Box-Plot: los box-plots resumen mínimo/máximo (bigotes), Q1 y Q3 (caja), mediana e $IQR = Q3 - Q1$. Regla típica: valores más allá de $\pm 1,5 \cdot IQR$ son outliers moderados y más allá de $\pm 3 \cdot IQR$ son severos. Atención: un scatter puede revelar puntos que un box-plot no destaca bien en ciertas configuraciones.
- Z-score (asumiendo gaussiana): mide cuántos desvíos estándar está una observación de la media. Regla de oro: $|Z| > 3$.
- Z-score modificado con MAD: como media y desvío estándar son sensibles a outliers, se usa la mediana y el MAD (mediana de desvíos absolutos). Para hacer MAD comparable al desvío estándar se normaliza por 0,6745. Regla de oro: $> 3,5$ indica outlier.

2.4 Precaución con el recorte de extremos

Una estrategia simple es eliminar los valores más altos y más bajos, pero ignora combinaciones de variables: puede fallar cuando la rareza surge de la interacción entre atributos.

2.5 Métodos multivariados: clustering, Mahalanobis, LOF e Isolation Forest

- Clustering / análisis globales: agrupan observaciones similares; puntos aislados respecto de los clusters pueden ser outliers.
- Distancia de Mahalanobis: mide la distancia de un punto respecto de la media del conjunto teniendo en cuenta la covarianza (usa la inversa de la matriz de covarianzas), corrigiendo por correlaciones entre variables. Valores grandes pueden indicar atipicidad.
- LOF (Local Outlier Factor): método basado en densidad. Compara la densidad local de un punto con la de sus k vecinos más cercanos (elegir k y métrica). Un score LOF = 5 significa que la densidad de los vecinos es 5 veces la del punto; se define un umbral de corte según el dominio.
- Isolation Forest: algoritmo no supervisado y no paramétrico basado en árboles. Idea: las anomalías se aíslan con pocas particiones. Procedimiento: tomar una muestra, elegir aleatoriamente n características y, para cada una, particionar con un valor aleatorio entre su mínimo y máximo; repetir recursivamente hasta aislar observaciones. Las anomalías tienen longitudes de ruta promedio cortas (altura del árbol pequeña).

3. Resumen integrador (cómo se conectan ambos temas)

- 1) En preprocesamiento, antes de modelar, se diagnostican faltantes, inconsistencias y outliers. La elección entre eliminar o imputar faltantes depende del mecanismo (MCAR/MAR/MNAR). Para imputar, hay desde reemplazos simples (media/mediana, con sus riesgos) hasta métodos multivariados como MICE o KNN.
- 2) La transformación ajusta la forma de los datos: normalizar (Min-Max, Z-score, decimal scaling), discretizar (igual frecuencia, igual ancho, cuantiles y reemplazos por estadísticos/etiquetas), aproximar normalidad (sqrt, log, Box-Cox), codificar categorías (dummies) y crear nuevas variables relevantes.
- 3) Los outliers se estudian con técnicas univariadas (IQR, Z-score, Z-modificado/MAD y box-plots) y multivariadas (clustering, Mahalanobis, LOF, Isolation Forest). No siempre se eliminan: a veces son el objetivo del análisis (fraudes/fallas). La decisión depende del problema y de cómo afectan al modelo.

Ejercicio 2

Explique brevemente el concepto de **Outliers** e indique cuál es la diferencia entre valores atípicos **univariados** y **multivariados**. Mencione métodos/técnicas para detectar estos valores en cada caso. Dé un ejemplo concreto de algún método utilizado en el TP1.

Los **outliers** o **valores atípicos** son observaciones distantes del resto de los datos, pueden deberse a un error de medición, aleatoriedad, etc. Siempre son subjetivos al problema que estamos estudiando y deben ser cuidadosamente inspeccionados porque pueden estar alertando anomalías.

Los **outliers univariados** son valores atípicos que podemos encontrar en una simple variable por ejemplo: en un conjunto de datos correspondientes a niños en edad escolar, encontramos una observación donde la edad es de 45 años. Métodos para detectarlos: IQR, Z-score, Z-score modificado.

Los **outliers multivariados** son valores atípicos que se pueden encontrar en un espacio n -dimensional, es decir analizando más de una variable o atributo. Por ejemplo en un dataset de frutas una manzana de color naranja. Métodos para detectarlos: Mahalanobis, LOF, Isolation Forest, Clustering.

EJEMPLO CONCRETO DE APLICACIÓN AL TP1

Outliers. Son observaciones que se desvían de manera inusual del patrón general y pueden distorsionar estadísticas y modelos; antes de eliminarlos, se justifican con criterio estadístico y de negocio.

Univariados vs. Multivariados.

- *Univariados*: extremos al mirar una variable.
- *Multivariados*: puntos “raros” solo al considerar varias variables en conjunto.

Detección univariada (ejemplos):

- Regla IQR (box-plot): límites inferior/superior = $Q1 - 1.5 \cdot IQR$ y $Q3 + 1.5 \cdot IQR$ (moderados); severos con $3 \cdot IQR$.
- Z-score: $Z = (x - \bar{x}) / s$; usualmente $|Z| > 3$.
- Z modificado (MAD): $Z_M = 0.6745 \cdot (x - \tilde{x}) / \text{MAD}$; umbral ≈ 3.5 (más robusto).

Detección multivariada (ejemplos):

- Distancia de Mahalanobis: $D^2 = (x - \mu)^T \Sigma^{-1} (x - \mu)$; comparar con percentiles de χ^2_k .
- Métodos de densidad/aislamiento: LOF e Isolation Forest (densidad local baja o rutas de aislamiento cortas).

Tratamiento típico: eliminar severos evidentes; *winsorizar/clip* los moderados; documentar reglas y verificar impacto en el modelo.

Ejemplo del TP1:

1. Construimos un vector con variables de viaje y montos (trip_distance, fare_amount, tip_amount, tolls_amount, extra).
2. Calculamos Mahalanobis y marcamos outliers por encima del $\sim 99.7\%$ (criterio tipo 3σ). Los removimos tras revisar dispersiones (distance vs fare).
3. Por variable aplicamos IQR: severos ($\pm 3 \cdot IQR$) se eliminaron; moderados ($\pm 1.5 \cdot IQR$) se winsorizaron.
4. Agregamos chequeos de coherencia: total_amount consistente y tope de negocio para montos extremos.

Conclusión breve: univariado = regla sobre una variable (IQR/Z); multivariado = estructura conjunta (Σ , densidad). En el TP1 combinamos Mahalanobis + IQR para detectar y tratar outliers de forma conservadora y justificada.

Ejercicio 1

Determine si las siguientes afirmaciones sobre los valores atípicos son verdaderas o falsas:

- El análisis de boxplot permite detectar outliers multivariados. _____
- Isolation Forest es un método basado en densidad que permite detectar tanto outliers univariados como multivariados. _____
- La única forma de tratar valores atípicos es eliminar los valores más altos y más bajos de una variable. _____
- Si se calcula el z-score para un conjunto de datos y se quieren eliminar los outliers deben eliminarse los valores dentro del rango -3 y 3. _____

“El análisis de boxplot permite detectar outliers multivariados.” → FALSO.

El boxplot resume una variable (mediana, Q1-Q3, IQR) y marca atípicos univariados. No capta rarezas que sólo aparecen por la combinación de varias variables.

“Isolation Forest es un método basado en densidad que permite detectar tanto outliers univariados como multivariados.” → FALSO.

Isolation Forest no es de densidad: aísla observaciones mediante particiones aleatorias; las rutas cortas indican anomalías. (Sí puede operar en espacios multivariados, pero su principio no es de densidad).

“La única forma de tratar valores atípicos es eliminar los valores más altos y más bajos de una variable.” → FALSO.

Hay varias opciones según el caso: winsorización/clip, transformaciones (log, Box-Cox), modelos robustos, revisión/corrección de errores, o mantenerlos si son válidos y relevantes; no es “sólo eliminar extremos”.

“Si se calcula el z-score... y se quieren eliminar los outliers deben eliminarse los valores dentro del rango -3 y 3.” → FALSO.

Con z-score se suelen marcar como atípicos los casos con $|z| > 3$ (fuera del rango $[-3, 3]$), no los que están dentro de ese rango.

Ejercicio 1

Determinar si las siguientes afirmaciones son verdaderas o falsas:

- La razón de la falta de datos (missing values) siempre es ajena a los datos en sí mismos. **F**
- La estandarización min-max lleva los valores de una variable al intervalo $[0, 1]$. **V**
- La estandarización z-score lleva los valores de una variable al intervalo $[0, 1]$. **F**
- Cuando se imputan datos faltantes por Media o Mediana la varianza estimada de la nueva variable no es válida porque está atenuada por los valores repetidos. **V**

“La razón de la falta de datos siempre es ajena a los datos en sí mismos.” → FALSO.

Existen mecanismos MCAR, MAR y MNAR: solo en MCAR la falta es completamente ajena; en MAR depende de otras variables observadas y en MNAR del propio valor faltante.

“La estandarización min-max lleva los valores al intervalo $[0, 1]$.” → VERDADERO.

Con $x' = (x - \min_{[i]}) / (\max_{[i]} - \min_{[i]})$ $x' = (x - \min) / (\max - \min)$ los valores quedan re-escalados a $[0, 1]$ (salvo casos degenerados como $\max_{[i]} = \min_{[i]}$ $\max = \min$ o si aparecen nuevos out-of-range).

“La estandarización z-score lleva los valores al intervalo $[0, 1]$.” → FALSO.

El z-score produce media 0 y desvío 1 ($z = (x - \bar{x}) / s_z = (x - \bar{x}) / s = (x - \bar{x}) / s$), no acota a $[0, 1]$; los z pueden ser negativos o mayores que 1 en módulo.

“Imputar faltantes por media o mediana atenúa la varianza; la estimación ya no es válida.” → VERDADERO.

Reemplazar por un único valor reduce la dispersión (apila observaciones en la media/mediana) y sesga a la baja la varianza y covarianzas, afectando inferencias.

Ejercicio 6

Una de las primeras tareas cuando se analiza un conjunto de datos es evaluar si existen valores atípicos.

En caso de encontrar algunos indique cuáles de las siguientes acciones serían correctas:

- Eliminarlos siempre del dataset para no interferir en el análisis. **_F_**
- Tratarlos con algunas de las técnicas utilizadas en datos faltantes y corregir su valor. **_F_**
- No se debe accionar sobre los valores atípicos, se deben conservar tal como están. **_F_**
- Analizarlos para identificar la fuente de los mismos antes de decidir si se eliminan o no. **_V_**

“Eliminarlos siempre del dataset...” → FALSO.

No existe regla de “eliminar siempre”: primero hay que diagnosticar la causa (error de carga, caso raro válido, cambio de régimen). Según el diagnóstico se decide.

“Tratarlos con técnicas de datos faltantes y corregir su valor.” → FALSO.

Un outlier no es un missing. En lugar de imputarlo como NA se usan enfoques específicos: winsorización/clip, transformaciones, métodos robustos o, si es error, corrección/eliminación.

“No se debe accionar... se deben conservar tal como están.” → FALSO.

A veces hay que actuar: corregir errores, recodificar, excluir extremos severos o aplicar modelos robustos. Dejar todo igual puede sesgar análisis y modelos.

“Analizarlos para identificar la fuente antes de decidir si se eliminan o no.” → VERDADERO.

Flujo recomendado: detectar → investigar origen y contexto → decidir tratamiento (conservar, transformar, winsorizar, eliminar) y documentar el criterio.

05- Árboles de Decisión: ID3, Entropía, Ganancia de Información, Impureza de Gini, C4.5 y Random Forest

1) ID3 (Iterative Dichotomiser 3)

ID3 (Ross Quinlan) genera un árbol de decisión a partir de ejemplos etiquetados. La salida es un grafo con forma de árbol que contiene:

- Un nodo raíz.
- Nodos internos (decisión), cada uno asociado a un atributo; sus aristas representan valores posibles de ese atributo.
- Nodos hoja (terminales) con la clase predicha.

2) Entropía de la información

La entropía mide impureza/aleatoriedad. Para un conjunto S con n clases y probabilidades $p_1, \dots, p_n$, la entropía (en bits) es:

$$H(S) = - \sum_{i=1..n} p_i \cdot \log_2(p_i)$$

Convenciones y propiedades:

- Si S es homogéneo (todas las instancias son de una sola clase), $H(S) = 0$.
- La entropía máxima es $\log_2(n)$. Si hay $n=2$ clases (True/False) y $p_1=p_2=0.5$, entonces $H(S)=1$.

3) Ganancia de Información (ID3)

La Ganancia de Información cuantifica cuánto reduce la entropía un atributo A al particionar el conjunto S . Se define como:

$$\text{Gain}(S, A) = H(S) - \sum_{v \in V(A)} \left(\frac{|S_v|}{|S|} \right) \cdot H(S_v)$$

donde $V(A)$ es el conjunto de valores del atributo A ; S_v es el subconjunto de S cuyo valor de A es v .

4) Algoritmo básico de ID3

- 1) Calcular $H(S)$ para la variable objetivo.
- 2) Para cada atributo A y cada uno de sus valores v , calcular $H(S_v)$ y el promedio ponderado $\sum \left(\frac{|S_v|}{|S|} \right) \cdot H(S_v)$.
- 3) Calcular $\text{Gain}(S, A)$ para todos los atributos y elegir el de mayor ganancia como nodo raíz (o subnodo, recursivamente).
- 4) Repetir recursivamente sobre los subconjuntos; detener al quedar homogéneo o sin atributos.

5) Ejemplo trabajado (tabla de animales)

Conjunto de entrenamiento (7 animales) con atributos: Vuela {sí/no}, Piel {pelo/pluma/escamas}, Nacimiento {útero/huevo}; clases: {Mamífero, Ave, Reptil}.

Tabla (resumen):

Vaca (no, pelo, útero) → Mamífero
Cerdo (no, pelo, útero) → Mamífero
Paloma (sí, pluma, huevo) → Ave
Murciélago (sí, pelo, útero) → Mamífero

Gallina (no, pluma, huevo) → Ave
Iguana (no, escamas, huevo) → Reptil
Cocodrilo (no, escamas, huevo) → Reptil

Distribución de clases en S: Mamífero=3/7, Ave=2/7, Reptil=2/7.

Entropía total:

$$H(S) = -[(3/7) \cdot \log_2(3/7) + (2/7) \cdot \log_2(2/7) + (2/7) \cdot \log_2(2/7)] \approx 1.557 \text{ bits.}$$

Atributo A = "Vuela". Valores: {sí, no}.

- S_no (5/7 del total): clases {Mamífero=2/5, Ave=1/5, Reptil=2/5}

$$H(S_{\text{no}}) = -[(2/5) \cdot \log_2(2/5) + (1/5) \cdot \log_2(1/5) + (2/5) \cdot \log_2(2/5)] \approx 1.522$$

- S_sí (2/7 del total): clases {Mamífero=1/2, Ave=1/2}

$$H(S_{\text{sí}}) = -[(1/2) \cdot \log_2(1/2) + (1/2) \cdot \log_2(1/2)] = 1.000$$

Promedio ponderado:

$$(5/7) \cdot H(S_{\text{no}}) + (2/7) \cdot H(S_{\text{sí}}) \approx (5/7) \cdot 1.522 + (2/7) \cdot 1.000 \approx 1.373$$

Ganancia:

$$\text{Gain}(S, \text{Vuela}) = H(S) - 1.373 \approx 1.557 - 1.373 \approx 0.184 (\approx 0.191 \text{ con redondeos de la lámina}).$$

De forma análoga se pueden calcular las ganancias para "Piel" y "Nacimiento". En la lámina aparecen (aprox.):

$$\text{Gain}(S, \text{Vuela}) \approx 0.191; \text{Gain}(S, \text{Piel}) \approx 1.557; \text{Gain}(S, \text{Nacimiento}) \approx 0.985.$$

El mejor atributo inicial es el de mayor ganancia.

6) Impureza de Gini

Otra medida de impureza (usada por varias implementaciones, p. ej. scikit-learn) definida como:

$$\text{Gini}(S) = 1 - \sum_{i=1..n} p_i^2$$

Tras un split por atributo A, la impureza ponderada es:

$$\text{Gini}_{\text{split}}(S, A) = \sum_{v \in V(A)} (|S_v|/|S|) \cdot \text{Gini}(S_v)$$

Se prefiere el atributo que minimiza la impureza ponderada del nodo raíz.

Interpretación de las láminas: se comparan varios árboles candidatos (por ejemplo, con atributos "Chest Pain", "Good Blood Circulation", "Blocked Arteries"). Se calcula Gini en cada hoja y luego el promedio ponderado en la raíz. El atributo con menor impureza total (en el ejemplo, "Good Blood Circulation") se elige como raíz.

7) C4.5: mejoras sobre ID3

C4.5 incorpora mejoras claves:

- Atributos numéricos (rangos continuos): transforma dinámicamente en pruebas booleanas $A < C$ vs $A \geq C$. Selección de umbral: ordenar ejemplos por A, detectar puntos donde cambia la clase, evaluar candidatos C_i y quedarse con el que maximiza la ganancia (o con los N mejores).
- Datos faltantes: permite "?"; al calcular ganancia/entropía, ignora registros con valor faltante de ese atributo.
- Poda del árbol: una vez construido el árbol, se analizan recursivamente nodos internos cuyos hijos son hojas. Se evalúa el error con conjunto de test; si al reemplazar el nodo por una hoja disminuye o no empeora el error, se elimina el nodo y sus hijos. (La lámina describe el "error" como Casos bien clasificados / Casos totales, es decir, precisión/accuracy sobre el conjunto de test).

8) Random Forest y Bagging

Idea central: combinar muchos estimadores débiles/mediocres y promediarlos para obtener un clasificador fuerte.

- Bagging (Bootstrap aggregating): a partir de un conjunto D de tamaño n , se generan m subconjuntos $D_1, \dots, D_m$ por muestreo aleatorio con reemplazo; en general se toma $n' \approx (2/3) \cdot n$ observaciones por subconjunto.
- Attribute bagging (random subspace): para cada subconjunto, seleccionar aleatoriamente un subconjunto de atributos; regla práctica: usar $\lfloor \sqrt{(\# \text{atributos})} \rfloor$ atributos.
- Entrenar un árbol por cada subconjunto/espacio y evaluar su desempeño (matriz de confusión). Tasa de error (según lámina):
$$\text{Error} = (\text{FP} + \text{FN}) / \text{TOTAL}$$

Seleccionar los mejores árboles y repetir hasta obtener K árboles.
- Clasificación de un nuevo ejemplo: votar con los K árboles; la clase mayoritaria es la salida final.

9) Recapitulación

- ID3 elige el atributo que más reduce la entropía (máxima Ganancia de Información).
- Gini es una alternativa eficiente para medir impureza; se elige el split con menor impureza ponderada.
- C4.5 mejora ID3 con atributos continuos, manejo de faltantes y poda.
- Random Forest aplica bagging + selección aleatoria de atributos para construir un conjunto de árboles y clasificar por voto mayoritario.

Ejercicio 3

Determinar si las siguientes afirmaciones sobre los árboles de decisión son verdaderas o falsas:

- El atributo que se utiliza en cada nodo para repartir las instancias es aquel que más aumenta la impureza. **F**
- ID3 es un algoritmo superior a C4.5 ya que permite trabajar con campos numéricos continuos. **F**
- El criterio de impureza de Gini es numéricamente similar al criterio de la entropía. **V**
- Cuando el árbol es demasiado profundo se evita el overfitting pero puede ser muy costoso computacionalmente, por ello suelen podarse. **F**

Falso. En cada nodo se elige el atributo que más reduce la impureza (máxima ganancia de información o mínima impureza de Gini), no el que la aumenta.

Falso. C4.5 es un avance sobre ID3: maneja atributos continuos (umbralizando), valores faltantes y poda; ID3 no. Por eso C4.5 no es “inferior” sino una mejora.

Verdadero (con matiz). Gini y Entropía no son iguales numéricamente, pero ambas miden impureza (mínimo en nodos puros y máximo cuando las clases están balanceadas) y suelen rankear splits de manera muy similar en la práctica; Gini, además, es más barato de calcular.

Falso. Profundizar mucho un árbol favorece el overfitting. Por eso se poda: para reducir complejidad, mejorar generalización y también abaratar el costo de evaluación.

Ejercicio 7

Indique cuáles de las siguientes afirmaciones son verdaderas:

- Los árboles de decisión se utilizan únicamente para problemas de clasificación. **F**
- La entropía es una medida de la impureza o aleatoriedad en los datos. En los árboles de decisión, buscamos que los nodos tengan la mayor entropía posible. **F**
- En árboles de decisión hay sobreajuste cuando el árbol es “demasiado” profundo. Una solución es detener el crecimiento del árbol antes que clasifique perfectamente a los datos. **V**
- En árboles de decisión hay sobreajuste cuando el árbol es “demasiado” profundo. Una solución es hacer crecer el árbol entero para que clasifique perfectamente a los datos y luego podar. **V**

Falso. Los árboles no son solo para clasificación: existen árboles de regresión para variables objetivo continuas.

Falso. La entropía mide impureza; al dividir buscamos reducirla (maximizar la ganancia de información). No se pretende “mayor entropía” en los nodos hijos, sino menor.

Verdadero. Un árbol “demasiado” profundo sobreajusta. Una forma de evitarlo es pre-poda: detener el crecimiento antes de que clasifique perfectamente (límites de profundidad, min_samples, umbrales de ganancia, etc.).

Verdadero. Otra estrategia es post-poda: dejar crecer el árbol (incluso hasta ajuste perfecto) y podar luego para reducir complejidad y mejorar generalización.

Ejercicio 6

Responda las siguientes preguntas en relación al tema: **Árboles de decisión**:

a. ¿Para qué utilizaría la técnica de **poda** en un árbol C4.5?

Se utiliza para controlar el overfitting: se hace después del entrenamiento, y consiste en “podar” o eliminar algunos nodos.

b. ¿Puede un árbol manejar valores de entrada numéricos? ¿Cómo?

Sí, dado un atributo A de rango continuo de valores, se debe establecer un umbral C que divida en dos el rango maximizando la ganancia de información. Con dicho umbral se creará un booleano Ac tal que si $A < C$ $Ac = TRUE$, sino $Ac = FALSE$.

c. ¿Para qué se usa la impureza de Gini? ¿Qué significa que un nodo sea puro o impuro?

Para determinar el nodo raíz y para generar las divisiones de los nodos del árbol, seleccionando el atributo que clasifica mejor. Para ello se mide la impureza de Gini. Un nodo puro tiene el 100% de instancias de la misma clase, un nodo impuro tiene instancias de diferentes clases.

a) Poda en C4.5 – para qué la usaría

Para evitar el sobreajuste y mejorar la generalización, simplificando el árbol. C4.5 hace post-poda: si el “costo/ error pesimista” estimado de un subárbol no mejora al de convertirlo en hoja (prediciendo la clase mayoritaria), lo reemplaza. Con eso reduce ramas causadas por ruido/valores atípicos y baja la complejidad del modelo.

b) ¿Puede manejar atributos numéricos? ¿Cómo?

Sí. C4.5 maneja variables continuas buscando un umbral t y separando en dos ramas: $x \leq t$ y $x > t$. Para hallarlo, ordena los valores distintos del atributo, evalúa puntos de corte candidatos entre valores adyacentes y elige el que maximiza la ganancia de información (o razón de ganancia). Así trata los numéricos sin discretizarlos previamente.

c) Impureza de Gini – para qué se usa y qué significa pureza/impureza

La Gini es una medida de impureza de un nodo: $G = 1 - \sum_k p_k^2$, donde p_k es la proporción de la clase k. Se usa para evaluar splits: buscamos particiones que reduzcan la impureza total (igual que con entropía).

- Nodo puro: todos los ejemplos son de una sola clase $\Rightarrow G=0$.
- Nodo impuro: hay mezcla de clases; cuanto más balanceadas, mayor Gini (máximo cuando las clases están equiprobables).

Ejercicio 9

Se tiene un conjunto de datos de 250 filas y 10 columnas y se quiere entrenar un árbol de decisión para resolver un problema de clasificación. Se genera un conjunto de entrenamiento (80%) y un conjunto de test (20%) y se quiere evaluar la performance del modelo en entrenamiento utilizando k-fold cross validation con k=5. Responder las siguientes preguntas:

- a) ¿Cuántas veces se entrenará el árbol de decisión? _____
- b) ¿Con cuántos registros se entrenará el árbol (en cada entrenamiento)? _____
- c) ¿Cuántos registros tendrá un conjunto de validación? _____
- d) ¿Cuántas veces se entrenaría el árbol si la proporción train/test fuera 70/30? _____

a) 5 veces.

Porque con k-fold CV con k=5 se entrena una vez por cada fold (k modelos), usando siempre el conjunto de entrenamiento (el test 20% queda aparte).

b) 160 registros por entrenamiento.

Total train = 80% de 250 = 200. En 5-fold, cada entrenamiento usa $k-1 = 4$ folds $\Rightarrow 4/5 \times 200 = 160$.

c) 40 registros en cada validación.

Cada fold de validación es $1/5$ del train $\Rightarrow 200/5 = 40$.

d) 5 veces.

La cantidad de entrenamientos en k-fold depende de k, no del tamaño del split. Con 70/30 y k=5, sigue siendo 5.

Ejercicio 10

Dado un conjunto de datos de entrenamiento se quieren optimizar los hiperparámetros de un árbol de decisión. Para ello se propone utilizar GridSearch Cross Validation

```
#parámetros a optimizar
params_grid = {'criterion':['gini','entropy'],
               'ccp_alpha':[0.001, 0.0173, 0.0336, 0.05],
               'max_depth':[2,4,5]}

#GridSearchCV
gridcv = GridSearchCV(estimator=arbol ,
                      param_grid=params_grid,
                      scoring='accuracy',
                      cv=10 )
```

Responder las siguientes preguntas:

a) ¿Cuántos juegos de parámetros se van a evaluar con el método GridSearchCV? _____

b) ¿Cuántas veces se va a evaluar cada set de parámetros? _____

c) Muestre dos ejemplos de juegos de parámetros evaluados

{ 'param1': valor, 'param2': valor, ..., 'paramN': valor }

1) _____

2) _____

d) ¿Qué representa el parámetro ccp_alpha? ¿Para qué se utiliza?

e) ¿Qué representa el parámetro max_depth? ¿Para qué se utiliza?

a) 24 juegos de parámetros

Por qué: GridSearchCV hace un producto cartesiano de los valores listados en param_grid. Tenés:

- criterion: 2 opciones (gini, entropy)
- ccp_alpha: 4 opciones (0.001, 0.0173, 0.0336, 0.05)
- max_depth: 3 opciones (2, 4, 5)
Entonces: $2 \times 4 \times 3 = 24 \times 4 \times 3 = 24$. No importa el orden, porque cada entrenamiento fija un valor por hiperparámetro y prueba todas las combinaciones posibles para no perder un óptimo potencial.

b) 10 veces cada set

Por qué: cv=10 indica validación cruzada 10-fold. Para cada combinación del punto (a), el grid parte el conjunto de entrenamiento en 10 folds, entrena con 9 y valida con el restante, rotando el fold de validación. Eso produce 10 puntajes (uno por fold) que luego se promedian. La idea es reducir la varianza de la estimación de desempeño y aprovechar mejor los datos, sin mirar el test externo.

c) Ejemplos de combinaciones evaluadas

1. {'criterion': 'gini', 'ccp_alpha': 0.001, 'max_depth': 2}
2. {'criterion': 'entropy', 'ccp_alpha': 0.0336, 'max_depth': 5}
Por qué: Son ejemplos válidos porque toman un valor de cada hiperparámetro listado. El grid va a recorrer también todas las demás (por ejemplo 'gini' con 0.05 y 4, etc.). Elegí estos dos para ilustrar extremos razonables: una configuración más regularizada por profundidad (depth=2, alpha pequeño) versus otra más expresiva (depth=5) pero con poda por complejidad (alpha intermedio).

d) Qué representa ccp_alpha y para qué se usa

ccp_alpha es el coeficiente de la poda por complejidad mínima (Cost-Complexity Pruning) de CART. Define el compromiso entre ajuste y tamaño del árbol minimizando:

$$R\alpha(T) = R(T) + \alpha \cdot |T|$$

donde $R(T)$ es el error (o una aproximación basada en impureza) y $|T|$ la complejidad (por ejemplo, número de hojas).

- Si $\alpha=0$, no penalizás la complejidad $\Rightarrow$ el árbol puede crecer y sobreajustar.
- Si α crece, cada hoja “cuesta más” $\Rightarrow$ se colapsan subárboles que no reducen suficientemente el error $\Rightarrow$ árbol más chico, mejor generalización.
Por qué lo tuneáramos: el α óptimo depende del ruido y la cantidad de datos. GridSearch lo prueba en varios valores porque pequeñas variaciones pueden cambiar cuántas ramas se podan y, por ende, el sesgo/varianza del modelo.

e) Qué representa max_depth y para qué se usa

max_depth fija la profundidad máxima del árbol (número de niveles desde la raíz hasta una hoja). Es una forma de pre-poda: limita la complejidad antes de crecer el árbol completo.

- Profundidad baja $\Rightarrow$ reglas más simples $\Rightarrow$ alto sesgo / bajo varianza $\Rightarrow$ riesgo de underfitting.
- Profundidad alta $\Rightarrow$ reglas muy específicas $\Rightarrow$ bajo sesgo / alta varianza $\Rightarrow$ riesgo de overfitting.
Por qué lo tuneáramos: porque el punto óptimo depende de cuán separables sean las clases y de la escala

del dataset. Al incluir `max_depth` en el grid, controlás explícitamente la complejidad estructural del árbol y equilibrás el trade-off sesgo-varianza junto con la poda de `ccp_alpha`. Combinarlos permite llegar a árboles compactos y precisos: `max_depth` evita crecer de más y `ccp_alpha` recorta lo que igualmente resulte innecesario tras el crecimiento.

06- Ensamblajes en ML: Introducción, AdaBoost, Gradient Boost, XGBoost y Ensamblajes Híbridos

1) Introducción a Ensamblajes

Idea base: combinar múltiples modelos para reducir varianza y mejorar generalización. Motivación por la “sabiduría de las multitudes”: el promedio/voto de varios modelos suele ser mejor que un único modelo, especialmente cuando los modelos individuales tienen errores distintos.

Bagging (Bootstrap aggregating): entrenar múltiples modelos sobre subconjuntos del train obtenidos por muestreo con reemplazo; promediar/votar al final. Beneficios: reduce varianza y puede atenuar el efecto de outliers (no aparecen en todos los subconjuntos). Problema con árboles: si hay pocos atributos predictivos fuertes, los árboles tienden a parecerse; Random Forest lo corrige seleccionando m atributos al azar en cada nodo.

Boosting: construir secuencialmente modelos que se enfocan en ejemplos mal clasificados por los previos; votación final ponderada. Beneficios: puede lograr alta exactitud; riesgo: sobreajuste si no se controla la capacidad.

1.1 Sesgo–Varianza y el trade-off

- Sesgo (bias): error por suposiciones/forma del modelo (modelos muy simples $\rightarrow$ underfitting).
- Varianza: sensibilidad a fluctuaciones del train (modelos muy complejos $\rightarrow$ overfitting).
- Ensamblajes reducen varianza (bagging) y/o controlan bias/varianza (boosting con tasa de aprendizaje, regularización).

2) AdaBoost (Adaptive Boosting)

Construye una secuencia de clasificadores débiles (stumps: árboles de un solo split) que corrigen los errores de los anteriores. Todos los ejemplos comienzan con igual peso y luego se re-ponderan según los aciertos/errores del clasificador actual.

2.1 Algoritmo (binario) y fórmulas

Inicialización: $w_1(i) = 1/N$ para $i=1..N$.

En la iteración t ($t=1..T$):

1) Entrenar $h_t(x)$ minimizando el error ponderado: $\epsilon_t = \sum_i w_t(i) \cdot \mathbb{1}\{h_t(x_i) \neq y_i\}$.

2) Ponderación (amount of say / peso del stump): $\alpha_t = \frac{1}{2} \cdot \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$.

3) Actualización de pesos por ejemplo i : $w_{t+1}(i) = w_t(i) \cdot \exp(\pm \alpha_t)$,

donde $+\alpha_t$ si h_t se equivoca en i , y $-\alpha_t$ si acierta; luego normalizar para que $\sum_i w_{t+1}(i)=1$.

Clasificador final: $H(x) = \text{sign}\left(\sum_{t=1..T} \alpha_t \cdot h_t(x)\right)$.

Interpretación de los gráficos/tablas: se compara qué stump (p. ej., “Patient Weight > 176”) logra menor impureza (Gini) con los pesos actuales; se calcula el error ponderado ϵ_t (ej., 1/8), luego $\alpha_t = \frac{1}{2} \ln((1-\epsilon_t)/\epsilon_t)$ (ej., $\frac{1}{2} \ln(7)=0.97$). Después se incrementan fuertemente los pesos de las instancias mal clasificadas multiplicando por $e^{+\alpha_t}$ (≈ 2.64) y se reducen los de las bien clasificadas por $e^{-\alpha_t}$ (≈ 0.38); finalmente se normaliza. Repetir T veces.

3) Gradient Boost (GBM)

Construye una “cadena” de árboles débiles (profundidad/hojas limitadas) que aprenden sobre los pseudo-residuos del modelo anterior. Cada árbol está ponderado por un learning rate (η). Funciona para regresión y clasificación.

3.1 Esquema y fórmulas

Objetivo general: minimizar una función de pérdida $L(y, F(x))$. Se parte de un modelo constante $F_0(x)$ (p. ej., media en regresión). Para $m = 1..M$:

- Calcular pseudo-residuos: $r_{\{im\}} = - [\partial L(y_i, F(x_i)) / \partial F(x_i)]_{\{F=F_{\{m-1\}}\}}$.
- Ajustar un árbol $h_m(x)$ a $\{ (x_i, r_{\{im\}}) \}$.
- Calcular la mejor escala γ_m resolviendo: $\gamma_m = \operatorname{argmin}_{\gamma} \sum_i L(y_i, F_{\{m-1\}}(x_i) + \gamma \cdot h_m(x_i))$.
- Actualizar: $F_m(x) = F_{\{m-1\}}(x) + \eta \cdot \gamma_m \cdot h_m(x)$.

Interpretación de gráficos/tabla de ejemplo (regresión del 'Weight'):

- Paso 1: F_0 = promedio (ej., 71.2 kg). Calcular residuos = $y - F_0$.
- Paso 2: Ajustar árbol a residuos con pocas hojas (ej., 4 hojas). Predecir residuos por hoja (promedios).
- Paso 3: Actualizar predicción parcial con learning rate (ej., $\eta=0.1$): $F_1 = F_0 + 0.1 \cdot h_1$. Residuo baja (mejora). Repetir con nuevos pseudo-residuos hasta M árboles o hasta que el error deje de mejorar.

3.2 Sobreajuste y control

- Learning rate pequeño ($\eta \approx 0.05-0.1$) + muchos árboles pequeños $\rightarrow$ suele generalizar mejor (varianza baja).
- Limitar profundidad/hojas por árbol; usar validación/early stopping.

4) XGBoost (eXtreme Gradient Boosting)

Implementación optimizada de Gradient Boost con regularización explícita y poda basada en 'ganancia' (Gain) y 'gamma' (γ). Trabaja con grandes datasets y aprovecha hardware. La explicación del material usa un ejemplo didáctico de regresión con residuos.

4.1 Objetivo regularizado y puntajes

Función objetivo aproximada (por iteración) con expansión de Taylor de segundo orden:

$$\text{Obj} \approx \sum_i [g_i \cdot f(x_i) + \frac{1}{2} h_i \cdot f(x_i)^2] + \Omega(f),$$

con $g_i = \partial L / \partial F|_{\{F_{\{prev\}}\}}$, $h_i = \partial^2 L / \partial F^2|_{\{F_{\{prev\}}\}}$, y regularización por árbol/hojas $\Omega(f) = \gamma \cdot T + \frac{1}{2} \lambda \sum_j w_j^2$.

- Peso óptimo de una hoja j : $w_j^* = -G_j / (H_j + \lambda)$, con $G_j = \sum_{\{i \in j\}} g_i$, $H_j = \sum_{\{i \in j\}} h_i$.
- 'Ganancia' de un split (izq/der): $\text{Gain} = \frac{1}{2} [G_L^2 / (H_L + \lambda) + G_R^2 / (H_R + \lambda) - G^2 / (H + \lambda)] - \gamma$.

La explicación de las láminas usa una versión simplificada con 'Similarity Score' = $(\sum \text{residuos})^2 / (n + \lambda)$, donde $\lambda \geq 0$ reduce varianza y hace caer puntajes/ganancias.

Poda: si $\text{Gain} - \gamma < 0$ en un nodo, se lo elimina. γ grande $\rightarrow$ árbol más conservador. Por defecto $\gamma=0$.

4.2 Procedimiento ilustrado (según láminas, regresión)

1) Predicción inicial $F_0 = 0.5$. 2) Calcular residuos (errores). 3) Árbol sobre residuos; Similarity Score del nodo = $(\sum \text{res})^2 / (n + \lambda)$. 4) Ganancia de split = $(\text{Score}_{\text{izq}} + \text{Score}_{\text{der}} - \text{Score}_{\text{raíz}})$. Elegir mejor umbral (p. ej., 'Dosage < 15'). 5) Repetir hasta profundidad fijada (p. ej., 2 niveles). 6) Poda con γ . 7) Actualizar F con learning rate ϵ (p. ej., 0.3): $F_1(x) = F_0(x) + \epsilon \cdot \text{output}_{\text{árbol}}(x)$, donde la salida de cada hoja $\approx (\sum \text{res}) / (n + \lambda)$. 8) Recalcular residuos y repetir.

4.3 Efecto de λ y γ (según gráficos)

- $\lambda > 0$ reduce Similarity Scores de manera proporcional al tamaño del nodo (más grande $\rightarrow$ mayor reducción relativa).
- Al bajar los Scores, también bajan las Gains; por tanto, con el mismo γ es más probable que haya poda.
- Con $\gamma > 0$ se eliminan splits con ganancia pequeña o negativa.

5) Ensamblados híbridos: Voting, Stacking, Cascading

Ensamblados homogéneos: combinan mismo tipo de modelo (bagging/boosting). Ensamblados híbridos: combinan clasificadores de distinto tipo.

5.1 Voting

Entrenar N modelos (potencialmente distintos) con los mismos datos y decidir por mayoría (o por voto ponderado si devuelven probabilidades). Reduce varianza; simple y robusto.

5.2 Stacking (Stacked Generalization)

Meta-aprendizaje: entrenar varios modelos base y luego un meta-modelo que, a partir de sus salidas (p. ej., probabilidades), aprende a combinarlos. Múltiples capas posibles. Meta-modelos típicos: árboles, Naive Bayes, SVM, perceptrón. Mejora sobre Voting cuando hay señales de “certeza” por clasificador.

5.3 Cascading

Secuencia de modelos (una sola ‘ruta’). Cada modelo procesa solo las instancias sobre las que el anterior no tuvo suficiente certeza. Útil cuando se requiere alta certeza en una clase (p. ej., detección de fraude):

- Ejemplo: si $P(\text{no fraude}) < 0.99$, pasar al siguiente modelo; si ninguno logra descartar fraude con alta confianza, se pasa a verificación humana.
- Suele ir de modelos simples a complejos; cascadas profundas pueden sobreajustar.

6) Propiedades, ventajas y desventajas

- Ventajas: en general mejor performance que modelos individuales, buen trade-off sesgo-varianza.
- Desventajas: menor interpretabilidad, mayor complejidad computacional.

7) Fórmulas clave (lista rápida)

AdaBoost:

$$\begin{aligned}\epsilon_t &= \sum_i w_t(i) \cdot \mathbb{1}\{h_t(x_i) \neq y_i\}; \quad \alpha_t = \frac{1}{2} \ln((1 - \epsilon_t) / \epsilon_t); \\ w_{t+1}(i) &= w_t(i) \cdot \exp(+\alpha_t) \text{ si } h_t(x_i) \neq y_i, \text{ o } \cdot \exp(-\alpha_t) \text{ si } \text{acierta}; \text{ normalizar}; \\ H(x) &= \text{sign}(\sum_t \alpha_t h_t(x)).\end{aligned}$$

Gradient Boost (genérico):

$$r_{\{im\}} = -\partial L / \partial F_{\{m-1\}}; \quad F_m = F_{\{m-1\}} + \eta \cdot \gamma_m \cdot h_m.$$

XGBoost (segundo orden):

$$\begin{aligned}w^*_j &= -G_j / (H_j + \lambda); \quad \text{Gain} = \frac{1}{2} [G_L^2 / (H_L + \lambda) + G_R^2 / (H_R + \lambda) - G^2 / (H + \lambda)] - \gamma; \\ \text{Similarity Score (versión didáctica):} &= (\sum \text{res})^2 / (n + \lambda).\end{aligned}$$

Ejercicio 3

Explique brevemente las principales diferencias entre:

- a. Métodos de ensamble de tipo **bagging** y los de tipo **boosting**.

Bagging

- Los conjuntos de entrenamiento se construyen mediante *bootstrap* (muestras aleatorias con reemplazo) para entrenar distintos modelos, y luego combinarlos.
- Los modelos se entrenan en paralelo.
- Disminuye la varianza en el modelo final
-

Boosting

- Se construyen nuevos modelos para las instancias mal clasificadas por los anteriores.
- Los modelos se entrenan secuencialmente.
- Las nuevas instancias se clasifican usando una votación ponderada de todos los modelos construidos.
-

- b. Métodos de ensamble de tipo **stacking** y los de tipo **voting**.

*Ambos son ensambles híbridos, se entrenan N modelos que pueden ser diferentes (árboles, KNN, Regresión Logística, SVM, etc). En **voting** se aplica votación mayoritaria o ponderada para decidir y en **stacking** se utiliza un metamodelo.*

a) Bagging vs. Boosting

- Dinámica de entrenamiento:
Bagging entrena muchos modelos *en paralelo* sobre distintos “bootstraps” (muestras con reemplazo) del mismo conjunto; luego agrega (promedio o voto).
Boosting entrena *secuencialmente*: cada modelo nuevo se enfoca en los casos que fallaron los anteriores y la combinación es ponderada.
- Efecto estadístico principal:
Bagging reduce varianza (promediar modelos inestables ↓ dispersión).
Boosting reduce sesgo (corrige errores iterativamente, volviendo al ensamble más “fuerte”).
- Práctica y riesgos:
Bagging es simple y naturalmente paralelizable (p. ej., Random Forest), con bajo riesgo de sobreajuste.
Boosting no se paraleliza fácilmente; si se lo “empuja” (muchas iteraciones, poca regularización), puede sobreajustar.

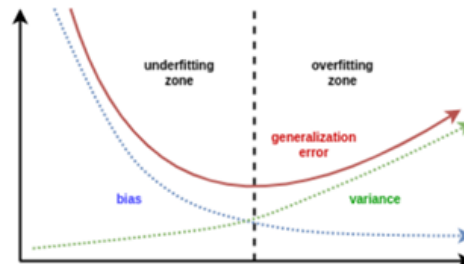
b) Stacking vs. Voting

- Regla de combinación:
Voting combina modelos (posible mezcla de tipos) con una regla fija: mayoría/voto ponderado; no se entrena un combinador.
Stacking entrena, además de los modelos base, un meta-modelo que aprende a combinar sus salidas (meta-aprendizaje).
- Cuándo usar:
Voting es apropiado cuando los modelos ya son buenos y diversos; se busca robustez simple.
Stacking es preferible cuando hay patrones en los errores de los modelos base que un meta-modelo puede explotar.
- Relación:
Voting puede verse como un caso particular de Stacking donde el “meta-modelo” es una regla fija (p. ej., promedio/majority).

Ejercicio 4

Explique los siguientes conceptos y cómo se relacionan con la **complejidad** de un modelo de aprendizaje automático: a) Error de Sesgo (**Bias**) b) Overfitting (**sobreajuste**)

- a) **Bias: modelos menos complejos tienen mayor sesgo.**
- b) **Overfitting: modelos más complejos aumentan la varianza**



a) El sesgo (bias) es el error sistemático que aparece cuando el modelo es demasiado simple para la relación real entre X e Y. Con baja complejidad (pocos parámetros, fuertes supuestos: linealidad, poca profundidad en árboles, regularización muy alta) el modelo no capta patrones clave: underfitting. Señales: error alto tanto en entrenamiento como en validación; residuales estructurados. Disminuye al aumentar la capacidad (más características/interacciones, menor regularización, modelos más flexibles).

b) El overfitting (sobreajuste) ocurre cuando un modelo de alta complejidad captura ruido idiosincrático del conjunto de entrenamiento. Consecuencia: error de entrenamiento muy bajo pero error de validación alto (alta varianza). Suele darse con muchos parámetros, profundidad excesiva en árboles, demasiadas iteraciones en boosting, o sin regularización/early stopping.

Relación con la complejidad: al incrementar complejidad, disminuye el sesgo pero aumenta la varianza; el error de generalización describe una curva en "U". Hay una zona de underfitting (modelo simple, alto sesgo), una zona de overfitting (modelo complejo, alta varianza) y un punto intermedio óptimo que se elige con validación (CV). Para mitigar sobreajuste: regularización (L1/L2), poda/pruning, early stopping, más datos, y ensambles que reduzcan varianza (p. ej., bagging/Random Forest).

Ejercicio 14

Explique brevemente las principales diferencias entre métodos de ensamble de tipo bagging y los de tipo boosting.

Bagging vs. Boosting

Bagging. Construye muchos modelos sobre muestras bootstrap del training set (muestras aleatorias con reemplazo) y luego los combina (promedio/voto). Su efecto principal es disminuir la varianza del modelo final y puede reducir el overfitting.

Boosting. Entrena secuencialmente: parte de un modelo base y, en cada iteración, entrena un nuevo modelo dando más importancia a las instancias mal clasificadas por los anteriores; la predicción final se hace con votación ponderada. En AdaBoost, este esquema depende del orden de construcción y no puede paralelizarse como en Random Forest; además, puede sobreajustar si se lo empuja demasiado.

En síntesis. Bagging = "muchos modelos en datos remuestreados para bajar varianza"; Boosting = "cadena de modelos que corrigen errores previos con voto ponderado", con mayor riesgo de sobreajuste si no se regula.

Ejercicio 4

Determinar si las siguientes afirmaciones son verdaderas o falsas:

- XGBoost sólo puede usarse en problemas de clasificación. F
- En Random Forest la votación reduce la varianza de la clasificación. V
- XGBoost maneja mejor el overfitting mediante regularizaciones. V
- Random Forest es muy útil para la selección de features. V

“XGBoost sólo puede usarse en problemas de clasificación.” → FALSO.

XGBoost soporta regresión (p. ej., reg:squarederror), clasificación binaria/multiclase y hasta ranking; no está limitado a clasificación.

“En Random Forest la votación reduce la varianza de la clasificación.” → VERDADERO.

RF es un bagging de árboles con aleatoriedad (bootstrap + submuestreo de features); al promediar/votar modelos poco correlacionados, la varianza del predictor conjunto disminuye.

“XGBoost maneja mejor el overfitting mediante regularizaciones.” → VERDADERO.

Incluye regularización L1/L2 de los pesos de hojas, shrinkage (learning rate), submuestreo de filas/columnas y límites de profundidad, todos mecanismos explícitos para controlar el sobreajuste.

“Random Forest es muy útil para la selección de features.” → VERDADERO.

Provee importancias de variables (impureza/permutación) que permiten priorizar o descartar atributos de forma práctica en pipelines de selección de características.

Ejercicio 5

Indique si las siguientes afirmaciones sobre **AdaBoost** y **Random Forest** son verdaderas o falsas (V/F):

- AdaBoost se entrena mediante la técnica de Bagging: _____
- En AdaBoost todos los árboles son completos mientras que Random Forest sólo entrena tocones (árboles con un solo nodo): _____
- En Random Forest cada árbol vota según su peso relativo: _____
- En AdaBoost se utiliza la técnica de Bootstrap aggregating para partir el dataset de entrenamiento en N datasets más pequeños: _____

a) Falso. AdaBoost no es bagging: entrena secuencialmente modelos débiles re-ponderando los ejemplos mal clasificados; no divide el set en bootstraps para entrenar en paralelo.

b) Falso. En AdaBoost los débiles learners típicos son tocon (decision stumps) o árboles muy poco profundos; en Random Forest se entrenan árboles profundos (de alta varianza) para luego agregarlos.

c) Falso. En Random Forest la regla estándar es voto no ponderado (cada árbol un voto; en regresión, promedio simple). No hay pesos relativos por árbol en el esquema básico.

d) Falso. AdaBoost no usa Bootstrap aggregating; usa reponderación (o muestreo proporcional a pesos) sobre el mismo conjunto de entrenamiento. Quien sí usa bootstrap/bagging para crear N subconjuntos es Random Forest.

Ejercicio 3

Determinar cuáles de las siguientes opciones son características que comparten los métodos de Bagging y Boosting. Indicar verdadero o falso:

- Se basan en entrenar varios modelos, y luego tomar una única decisión. _____
- Cada modelo se entrena en paralelo. _____
- Cada modelo es influenciado por la performance de los otros. _____
- Utilizan datasets de entrenamiento generados mediante Bootstrap. _____

“Se basan en entrenar varios modelos y luego tomar una única decisión.” → V

Ambos construyen múltiples modelos y luego agregan sus salidas (promedio/voto/ponderación) para una sola predicción.

“Cada modelo se entrena en paralelo.” → F

Es cierto para bagging (paralelizable), pero boosting es secuencial; por lo tanto, no es una característica compartida.

“Cada modelo es influenciado por la performance de los otros.” → F

En boosting sí (cada iteración corrige errores previos), en bagging no (modelos independientes). No la comparten.

“Utilizan datasets de entrenamiento generados mediante bootstrap.” → F

Bagging usa bootstrap explícito; boosting re-pondera/remezcla ejemplos por pesos, no hace bagging clásico. No es común a ambos.

07- K-Nearest Neighbors (KNN) y Support Vector Machines (SVM)

1) K-Nearest Neighbors (KNN)

KNN es un método de clasificación (y regresión) basado en vecindad. Dado un nuevo ejemplo x^* , se buscan sus k vecinos más cercanos en el conjunto de entrenamiento y se decide por votación (clasificación) o promedio (regresión).

1.1 Intuición con gráficos (Iris y puntos de colores)

- Caso 1 (vecino más cercano): si x^* está más cerca (por distancia Euclídea) de puntos 'verdes', se asigna a la clase 'verde'.
- Caso 2 ($k=7$): si los 7 vecinos más cercanos son todos verdes, x^* será 'verde'.
- Caso 3 ($k=9$): si hay 5 vecinos verdes y 4 amarillos, gana 'verde' por mayoría simple.
- Conjuntos desbalanceados: si hay 6 azules y 3 rojos entre los 9 vecinos, la clase mayoritaria 'azul' domina.
- Outliers: si $k=3$ y alrededor de x^* hay 3 rojos aislados (outliers locales), la votación da 'rojo'.

Los gráficos originales muestran estos escenarios variando la posición de x^* , el valor de k y las proporciones de clases.

1.2 Distancias y voto (fórmulas)

- Distancia Euclídea entre $x=(x_1,...,x_d)$ y $y=(y_1,...,y_d)$:
$$d_2(x, y) = \sqrt{\sum_{j=1..d} (x_j - y_j)^2}$$
- Distancia Manhattan (L_1):
$$d_1(x, y) = \sum_{j=1..d} |x_j - y_j|$$
- Distancia de Minkowski (orden $p \geq 1$):
$$d_p(x, y) = (\sum_j |x_j - y_j|^p)^{1/p}$$
- Votación no ponderada (clasificación):
$$\hat{y}(x^*) = \arg \max_c \sum_{i \in N_k(x^*)} \mathbb{1}\{y_i = c\}$$
- Votación ponderada por distancia (más cercanos pesan más):
$$\hat{y}(x^*) = \arg \max_c \sum_{i \in N_k(x^*)} w_i \cdot \mathbb{1}\{y_i = c\}, \text{ con } w_i = 1 / (d(x^*, x_i)^p + \epsilon)$$

(p típico entre 1 y 2; $\epsilon > 0$ evita división por cero).
- Regresión KNN (para completitud): $\hat{y}(x^*) = (1/k) \cdot \sum_{i \in N_k(x^*)} y_i$ (no ponderado), o $\hat{y}(x^*) = \sum w_i y_i / \sum w_i$.

1.3 Hiperparámetros y efectos

- k : pequeño $\rightarrow$ alta varianza (sensibilidad a ruido/outliers); grande $\rightarrow$ alto sesgo (promedia en exceso).
- Métrica de distancia: Euclídea/Manhattan/Minkowski (elegir según escala y naturaleza de las features).
- Peso por distancia: mitiga votaciones sesgadas cuando hay vecinos a distancias muy distintas.
- Escalado: imprescindible si las variables tienen unidades muy diferentes (Estandarización o Min-Max).

Sensibilidades destacadas en el material: KNN es sensible a conjuntos no balanceados (la mayoría gana por votación) y a outliers (vecinos anómalos pueden arrastrar la decisión si k es chico). En scikit-learn, $k=5$ es el valor por defecto.

2) Support Vector Machines (SVM)

SVM busca un clasificador de margen máximo (o margen blando) que separe clases con el mayor margen posible. Los ejemplos que "soportan" el margen se denominan vectores de soporte (support vectors).

2.1 Intuición con gráficos (ratones: masa/altura, umbrales y margen)

- En 1D (masa de ratones), un clasificador ‘umbral’ separa obesos vs no-obesos. Ubicarlo al azar puede ser malo. El ‘clasificador de margen máximo’ coloca el umbral en el punto medio entre las observaciones límite de cada clase, maximizando la distancia mínima (margen) a los datos.
- Sensibilidad a outliers: un outlier extremo movería el umbral. Para robustecer, se permite cierta cantidad de clasificaciones erróneas → margen blando (soft margin).
- En 2D (masa vs altura) el límite es una línea; en 3D un plano; en d dimensiones, un hiper-plano.

2.2 Formulación (lineal) y margen blando (fórmulas)

Clasificación binaria con etiquetas $y_i \in \{-1, +1\}$. Hiper-plano: $f(x) = w^T x + b$.

- Margen geométrico (para datos separables): $2 / \|w\|$.
- Problema de optimización (margen blando, $C > 0$ controla penalización por error):

$$\min_{\{w, b, \xi\}} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i$$

s.a. $y_i (w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$

La constante C balancea errores vs. tamaño del margen: C grande → menos errores (más rigidez, mayor varianza); C chico → más tolerancia a errores (mayor sesgo, menor varianza).

- Regla de decisión: $\hat{y}(x) = \text{sign}(w^T x + b)$.
- Vectores de soporte: puntos que tocan o caen dentro del margen (o lo violan), determinan w y b.

2.3 No linealidad y el truco del kernel

Cuando la frontera no es lineal (p. ej., curados vs no-curados según ‘dosis’), la idea es proyectar los datos a una dimensión mayor donde sí sean separables y aprender allí un Support Vector Classifier.

El ‘truco del kernel’ evita calcular explícitamente esa proyección: basta con definir un kernel $K(x, z)$ que compute el ‘producto interno’ en el espacio de mayor dimensión.

- Kernel polinómico (grado d): $K(x, z) = (x^T z + c)^d$ ($c \geq 0$). Aumenta la dimensionalidad de forma controlada; se elige d por validación (como ilustra el material).
- Kernel radial (RBF/Gaussiano): $K(x, z) = \exp(-\gamma \cdot \|x - z\|^2)$. Se interpreta como un clasificador que pondera más los vecinos cercanos y atenúa los lejanos; γ controla la ‘anchura’ de influencia local.
- Clasificador en forma dual (intuitivo): $f(x) = \text{sign}(\sum_i \alpha_i y_i K(x_i, x) + b)$, con $\alpha_i \geq 0, y_i \in \{-1, +1\}$. Los α_i no nulos corresponden a vectores de soporte.

2.4 Soft margin en la práctica (explicación de los gráficos)

Los gráficos muestran cómo, permitiendo cierto número de errores dentro del margen, se logra una frontera más estable ante outliers y con mejor generalización. El ‘mejor’ margen blando (valor de C y, si aplica, del kernel) se elige por validación cruzada (k-fold o particiones 70/30), comparando el desempeño en el conjunto de validación.

2.5 Dimensionalidad y forma del límite

- 1D: el límite es un punto (umbral).
- 2D: línea.
- 3D: plano.
- dD: hiper-plano (dimensión d-1).

El material muestra ejemplos visuales para 1D, 2D y 3D (masa, altura, edad), y comenta la dificultad de ‘ver’ 4D+.

2.6 Sesgo-varianza (según las láminas)

- Umbral muy sensible al entrenamiento → bajo sesgo, alta varianza (sobreajuste).
- Umbral menos sensible → mayor sesgo, baja varianza (mejor generalización). El margen blando (C y kernel) permite ajustar ese trade-off.

Ejercicio 4

Indique a qué conjunto (CRUZ o CÍRCULO) pertenece el punto negro según el parámetro K en el modelo de **K-Nearest-Neighbors**:



- a) K = 1 _____
b) K = 4 _____
c) K = 8 _____

a) K = 1 → CRUZ

Justificación: con 1-NN la clase se decide por el único vecino más cercano. El punto negro tiene como vecino más próximo la cruz que está inmediatamente a su izquierda, por lo que se clasifica como CRUZ (regla de vecino más cercano / voto simple).

b) K = 4 → CÍRCULO

Justificación: tomamos los 4 vecinos más cercanos al punto negro. Son: 1 cruz (la que está a su izquierda) y 3 círculos (arriba-derecha, derecha y abajo-derecha). Por mayoría 3-1 el voto es CÍRCULO.

c) K = 8 → CRUZ

Justificación: con K=8 entran todos los puntos del dibujo (5 cruces en total: 1 cercana + 4 del grupo de la izquierda; y 3 círculos a la derecha). La mayoría 5-3 favorece a la clase CRUZ.

Observaciones (ligadas a la teoría K-NN)

- La regla de K-NN clasifica por votación de los K vecinos más cercanos (típicamente con distancia euclídea) y la clase ganadora es la de mayoría simple.
- Al variar K cambia la vecindad “visible” por el punto negro: con K pequeño domina el vecino inmediato (aquí, la cruz), con K intermedio puede imponerse el pequeño grupo local de la otra clase (K=4 → círculos), y con K grande influye la distribución global (K=8 → cruz por mayoría global).

Ejercicio 2

Indique si las siguientes afirmaciones sobre sobre **SVM** son verdaderas o falsas (V / F) :

- a) SVM es un algoritmo que permite buscar un *Support Vector Classifier* (un clasificador de margen blando) en un espacio de dimensión más pequeña, que el espacio de entrada original: _____
- b) SVM, a través de uno de sus Kernels es capaz de encontrar soluciones en un espacio de dimensiones infinitas: _____
- c) SVM **no** realiza un mapeo real del espacio de entrada al nuevo espacio, sino que solo computará las relaciones entre pares de observaciones como si estuviesen en el nuevo espacio, lo que comúnmente se llama, *kernel trick*: _____
- d) La función Kernel polinómica, necesita un parámetro **d**, llamado grado del polinomio que indica el grado más pequeño del polinomio a usar: _____

a) Falso.

La idea central de SVM (con kernels) es llevar los datos a una dimensión mayor y allí buscar un Support Vector Classifier que separe los grupos; no a una menor.

b) Verdadero.

Con el kernel radial (RBF), SVM “soporta Support Vector Classifiers en infinitas dimensiones”.

c) Verdadero.

El kernel trick consiste en no mapear explícitamente al nuevo espacio: solo se calculan relaciones entre pares como si los puntos estuvieran en ese espacio, evitando la transformación explícita.

d) Falso.

El kernel polinómico usa un parámetro **d** que es el grado del polinomio (y determina cuántas dimensiones se agregan); no “el grado más pequeño” a usar.

Notas de teoría usadas: SVM incrementa dimensionalidad para separar (idea central), RBF permite trabajar en dimensión infinita, el kernel trick evita el mapeo explícito y el kernel polinómico ajusta la dimensionalidad mediante su grado **d**.

08 - Reducción de Dimensionalidad (PCA, MDS/PCoA, t-SNE e ISOMAP)

1) PCA — Análisis de Componentes Principales

Objetivo: proyectar datos de alta dimensión a un subespacio de menor dimensión maximizando la varianza preservada. Aplicaciones: visualización 2D/3D, compresión, reducción de ruido, acelerar modelos y presentación de resultados.

1.1 Intuición geométrica (gráficos de la recta ‘mejor ajustada’)

Se centra el conjunto de datos y se busca la recta por el origen que mejor ‘ajusta’ a los puntos. Por Pitágoras, si $a^2 = b^2 + c^2$ y ‘a’ es constante aunque rote la recta, minimizar b^2 equivale a maximizar c^2 . PCA maximiza la suma de distancias al cuadrado desde las proyecciones de los puntos sobre la recta hasta el origen.

En los slides, se etiqueta cada distancia proyectada como $d_1, \dots, d_6$ y se muestra explícitamente que PCA maximiza $d_1^2 + d_2^2 + d_3^2 + d_4^2 + d_5^2 + d_6^2$ (la ‘suma de distancias cuadradas’). Ese valor coincide con el autovalor asociado a PC1.

1.2 PC1, pendiente y cargas (‘loading scores’)

En el ejemplo didáctico con ‘árboles’ (eje X) y ‘RNA’ (eje Y), la pendiente de PC1 es 0.25: por cada 4 unidades en ‘árboles’, 1 unidad en ‘RNA’. Esto sugiere mayor dispersión en X que en Y. PC1 es una ‘combinación lineal de variables’. Si el vector dirección de PC1 antes de normalizar es $(b, c) = (4, 1)$, su longitud es $a = \sqrt{(4^2 + 1^2)} = \sqrt{17} \approx 4.12$. Al normalizar por SVD se obtiene el autovector con norma 1: $v_1 = (4/4.12, 1/4.12) \approx (0.97, 0.241)$. Estas son las ‘loading scores’ para PC1.

PC2 en 2D es la recta perpendicular a PC1 (p. ej., vector proporcional a $(-1, 4)$ normalizado). El autovalor de PC2 es la suma de distancias cuadradas proyectadas sobre PC2, análogo a PC1.

1.3 Cálculo matricial (SVD) y varianza explicada

Dados datos centrados X ($m \times p$), la descomposición $X = U \Sigma V^T$ produce $V = [v_1, \dots, v_p]$ (autovectores) y autovalores proporcionales a $\Sigma^2 / (m-1)$. Las proyecciones en k dimensiones: $Z = X \cdot V_k$. La ‘varianza explicada’ por la j -ésima PC es $\lambda_j / \sum_i \lambda_i$. En el gráfico de ejemplo, PC1 explica $\approx 91\%$ y PC2 $\approx 4\%$ (scree plot o anotación en ejes).

2) MDS (Multi-Dimensional Scaling) y PCoA

Idea central: preservar distancias entre pares de puntos al reubicar los datos en menor dimensión. Clásicamente, MDS y PCoA son equivalentes; los slides lo remarcan explícitamente.

2.1 Ejemplo guiado (matriz de puntajes y matriz de distancias)

Se parte de la tabla de ‘puntajes por tema’ (A1..A6) con cuatro columnas: árboles, RNA, PLN y SVM. Se calcula la distancia entre cada par (A1-A2, A1-A3, etc.) usando, por ejemplo, Euclídea. En el slide se muestra: $\|A1-A2\| = \sqrt{[(9-10)^2 + (6-4)^2 + (12-9)^2 + (5-7)^2]} = \sqrt{[1 + 4 + 9 + 4]} = \sqrt{18} \approx 4.24$. Se arma la matriz de distancias simétrica (diagonal 0).

Luego se busca una matriz M de coordenadas 2D (MDS1, MDS2) tal que las distancias euclídeas allí se acerquen lo más posible a la matriz de distancias original. Es decir, si $A_1=(x_1, y_1)$, $A_2=(x_2, y_2)$, entonces $\|A_1-A_2\|_{\text{MDS}} = \sqrt{[(x_1-x_2)^2 + (y_1-y_2)^2]} \approx 4.24$, etc.

2.2 Función objetivo ‘Stress’ y algoritmos (descenso/SMACOF)

Se define la función de error ‘Stress’ como la suma (o raíz de la suma) de las diferencias entre distancias originales y reducidas. Se minimiza por métodos iterativos: descenso por gradiente (Kruskal) o mayorización (SMACOF, De Leeuw). Los slides muestran el procedimiento con valores concretos, actualizando coordenadas y reduciendo un error agregado (p. ej., de ≈ 4.05 a 4).

En un ejemplo con 4 observaciones (A1..A4), se fijan coordenadas iniciales aleatorias y se calculan distancias entre los puntos en 2D; luego se computa el error global respecto de la matriz objetivo y se mueven los puntos en la dirección del gradiente negativo para bajar el Stress. Este proceso se repite hasta estabilizar.

2.3 Métricas alternativas y observaciones

MDS permite distintas métricas (Manhattan, Hamming, Gran-círculo, Mahalanobis). Minimizar distancias lineales es análogo a maximizar correlación lineal en el embedding; no obstante, la métrica elegida puede resaltar otras estructuras. Resultado de la demo: se obtiene un mapa 2D muy similar al de PCA para el conjunto de puntajes.

3) t-SNE (t-Distributed Stochastic Neighbor Embedding)

Objetivo: visualizar en menor dimensión preservando relaciones de vecindad y la estructura de clusters. El procedimiento es secuencial (‘mover puntos de a uno’), con fuerzas de atracción por vecinos cercanos y repulsión por puntos lejanos.

3.1 Similitudes en alta dimensión y ‘perplejidad’

Para cada punto x_i se calcula su similitud con otros puntos usando una Gaussiana centrada en x_i . El ancho (desvío estándar) NO es fijo: depende de la densidad local y se sintoniza para que la ‘perplejidad’ —número efectivo de vecinos— sea el valor elegido por el usuario. Cuanto mayor la perplejidad, más vecinos ‘relevantes’ se consideran. Las probabilidades condicionales $p_{\{j|i\}}$ se normalizan (se divide por la suma de similitudes) y luego se simetrizan a $p_{\{ij\}}$ promediando ambas direcciones.

3.2 Similitudes en baja dimensión (t-Student) y objetivo

En el espacio reducido se definen similitudes $q_{\{ij\}}$ con una distribución t-Student (colas más pesadas que la Normal) para evitar el ‘crowding’ y separar mejor grupos. Conceptualmente, t-SNE ajusta posiciones para que la matriz q se parezca a la matriz p . En la práctica se optimiza (vía gradiente) una divergencia entre ambas (los slides explican la intuición con ‘hacer coincidir filas’ moviendo puntos de a poco). Se subraya que el método es estocástico, costoso y no ofrece mapeo para puntos nuevos.

3.3 Lectura de los gráficos de la secuencia

Los slides muestran: (1) puntos en 2D proyectados a 1D; (2) inicio con orden aleatorio; (3) ‘atracción’ por vecinos del mismo cluster y ‘repulsión’ por puntos distantes; (4) cálculo de similitudes con Gaussiana de ancho adaptativo; (5) normalización por la suma; (6) simetrización $p_{\{ij\}} = (p_{\{j|i\}} + p_{\{i|j\}})/2$; (7) en baja dimensión, cálculo de similitudes con t-Student; (8) actualización iterativa de posiciones para que cada fila ‘se parezca’ a la original. Finalmente, se exhiben comparativas t-SNE vs PCA (tiempos del ejemplo: ~ 9 s vs ~ 0.03 s) y un caso MNIST.

4) ISOMAP (Isometric Mapping)

Hipótesis de variedades: los datos reales de alta dimensión suelen ‘vivir’ cerca de una variedad de baja dimensión. Si los datos en $\mathbb{R}^3$ siguen una forma ‘S’, la distancia Euclídea entre dos extremos puede ser corta, pero la distancia a lo largo de la ‘S’ (geodésica) es grande. ISOMAP busca preservar esas distancias geodésicas.

4.1 Algoritmo (paso a paso con los gráficos)

- 1) Construir un grafo k-NN: para cada punto, unir con sus K vecinos más cercanos por distancia Euclídea; ponderar aristas con esa distancia (k es hiperparámetro).
- 2) Calcular distancias más cortas entre todos los pares en el grafo (Dijkstra o Floyd-Warshall) para aproximar las distancias geodésicas; formar la matriz G.
- 3) Aplicar MDS/PCoA sobre G para obtener la proyección en baja dimensión.

Los slides enfatizan la diferencia Euclídea vs Geodésica con dibujos: líneas rectas punteadas vs caminos sobre la superficie ('S'), y muestran la proyección deseada que 'desenrolla' la variedad.

4.2 Debilidades y 'Landmark-ISOMAP'

Debilidades: (a) es lento a partir de ~1000 observaciones; (b) k muy grande puede puentear regiones o amplificar ruido; (c) k muy chico puede desconectar el grafo y arruinar geodésicas; (d) puntos mal medidos distorsionan muchas entradas de la matriz. Landmark-ISOMAP: seleccionar N 'landmarks' (uniformemente, $N < \text{total}$), calcular distancias solo respecto de ellos y luego usar Landmark-MDS (LMDS) para obtener la proyección de todos los puntos de forma más eficiente.

5) Comparativa y guía de uso

- PCA: lineal, muy rápido; interpretar con varianza explicada y 'loadings'.
- MDS/PCoA: preserva distancias con una métrica elegida; requiere optimización iterativa (descenso/SMACOF).
- t-SNE: gran visualizador de clusters locales; perplejidad (vecinos efectivos) es clave; estocástico; sin mapeo out-of-sample.
- ISOMAP: 'desenrolla' variedades preservando geodésicas; sensible a k y al ruido; Landmark-ISOMAP para escalar.

Ejercicio 5

Responda las siguientes preguntas en relación al tema: **Reducción de Dimensionalidad:**

- a. ¿Para qué usaría un gráfico "Scree plot", al utilizar PCA?

Para seleccionar el número de componentes principales, utilizando el método "elbow", de acuerdo al porcentaje de varianza explicada.

- b. ¿Qué técnica utilizaría si sospecha que los datos están distribuidos sobre una **variedad** del espacio total?

ISOMAP: Su objetivo es preservar la geometría intrínseca de los datos reflejada en las distancias geodésicas de la variedad. La clave es encontrar una forma eficiente de calcular la verdadera distancia geodésica entre observaciones, dada solamente su distancia Euclídea en el espacio de alta dimensión

- c. ¿Qué técnica utilizaría si desea conservar los **clusters**?

TSNE, ya que la proyección de t-SNE preserva los clusters.

a) Uso del "Scree plot" en PCA

Se utiliza para decidir cuántas Componentes Principales retener. El scree plot muestra el porcentaje de variación explicado por cada PC; al observar el "codo" (punto donde el aporte marginal cae bruscamente) se elige k de forma que se conserve la mayor parte de la variabilidad con el menor número de componentes, p.ej. "PC1 83% - PC2 17%... Scree plot representa el porcentaje de variación de cada PC".

b) Si los datos viven sobre una variedad (manifold)

ISOMAP. Parte de la hipótesis de variedades y reemplaza la distancia euclidiana por la geodésica (distancia a lo largo de la variedad), logrando una proyección que respeta la topología intrínseca del dato. Operativamente: (1) grafo de k-vecinos, (2) distancias de camino más corto (Dijkstra/Floyd-Warshall), (3) aplicar MDS sobre esa matriz de distancias.

Justificación: cuando hay curvatura (p. ej., “forma de S”), las distancias euclidianas confunden puntos lejanos sobre la variedad; la geodésica recupera la estructura real.

c) Si se desea conservar los clusters

t-SNE. Es un método de proyección no lineal diseñado para preservar los clusters (estructura local y también global en muchos casos) en espacios de baja dimensión, siendo especialmente efectivo para visualización.

Justificación: define similitudes entre puntos en el espacio original y las hace corresponder en el espacio reducido (usando distribuciones Gauss/t-Student y un proceso iterativo), produciendo agrupamientos visibles y separados.

Ejercicio 1

Indique qué método de **reducción de la dimensionalidad** (PCA, MDS, t-SNE, ISOMAP) aplica mejor según el caso:

- a) Se sospecha que los datos están ubicados en una variedad, dentro del espacio de alta dimensionalidad: _____
- b) Se privilegia preservar las distancias entre puntos, incluso para métricas no euclidianas: _____
- c) Se requiere entender cuán dispersos están los datos y sobre todo, sobre qué ejes o variables: _____
- d) Se requiere proyectar un espacio de alta dimensionalidad en tan solo 2 dimensiones, manteniendo los clusters del conjunto original: _____

a) ISOMAP.

Cuando se sospecha que los datos yacen sobre una variedad (manifold) no lineal, ISOMAP reemplaza las distancias euclidianas por distancias geodésicas (vía grafo de k-vecinos + caminos más cortos) y luego aplica MDS sobre esa matriz, preservando la geometría intrínseca.

b) MDS (Escalamiento Multidimensional).

Su objetivo es encontrar una configuración en baja dimensión que preserve lo mejor posible las distancias/diferencias entre puntos a partir de una matriz de disimilitudes, incluso cuando éstas no son euclidianas (p. ej., métricas arbitrarias o no métricas).

c) PCA.

Para entender cuán dispersos están los datos y sobre qué ejes se explica esa dispersión, PCA busca direcciones ortogonales que maximizan la varianza y entrega autovalores (varianza explicada) y autovectores (loadings) que permiten interpretar qué variables definen cada eje.

d) t-SNE.

Si se requiere proyectar a 2D manteniendo visiblemente los clusters, t-SNE preserva vecindades locales (altas similitudes en el espacio original siguen siendo altas en la proyección) y produce agrupamientos bien separados para fines de visualización.

09 - Redes Neuronales

1) Perceptrón, Función Escalón y AND

Función escalón de Heaviside:

$$H(x) = 1 \text{ si } x \geq 0, \text{ y } 0 \text{ en otro caso.}$$

Perceptrón binario (dos entradas): dado $x = (x_1, x_2)$, pesos $w = (w_1, w_2)$ y sesgo b , la activación lineal es

$$z = x_1 \cdot w_1 + x_2 \cdot w_2 + b$$

y la salida con escalón es $\hat{y} = H(z)$.

Tabla de verdad AND (etiquetas): $(0,0) \rightarrow 0, (0,1) \rightarrow 0, (1,0) \rightarrow 0, (1,1) \rightarrow 1$.

Ejemplo de entrenamiento (del material): inicialización aleatoria $w_1=0.3, w_2=0.2, b=-1$.

Primera iteración: $z(1,1)=0.3+0.2-1=-0.5 \Rightarrow \hat{y}=0, y=1 \Rightarrow \text{error}=1$.

Regla del perceptrón (tasa α): $w \leftarrow w + \alpha \cdot (y - \hat{y}) \cdot x, b \leftarrow b + \alpha \cdot (y - \hat{y})$.

Con $\alpha=0.2$ (como en las láminas): tras 2 actualizaciones, en la 3ª iteración $w=(0.7,0.6), b=-1$, y $z(1,1)=0.3 \Rightarrow \hat{y}=1$. El modelo entrenado queda $(w_1, w_2, b)=(0.7, 0.6, -1)$.

1.1 Limitaciones y MLP

El perceptrón resuelve separabilidad lineal pero no XOR. El Perceptrón Multicapa (MLP) introduce capas ocultas con activaciones no lineales.

2) Backpropagation — ejemplo numérico

Arquitectura: 2 entradas ($i_1=0.05, i_2=0.10$), 2 ocultas (h_1, h_2) y 2 salidas (o_1, o_2), objetivos (0.01, 0.99). Activación sigmoide en ocultas y salida.

Sigmoide y derivada: $\sigma(a)=1/(1+e^{-a})$, $\sigma'(a)=\sigma(a) \cdot (1-\sigma(a))$.

Forward: $a_h = W^1 x + b^1, h = \sigma(a_h)$; $a_o = W^2 h + b^2, o = \sigma(a_o)$. Resultados (lámina): $h \approx (0.59327, 0.59688)$; $o \approx (0.75137, 0.77290)$.

Error cuadrático medio por salida: $E_j = \frac{1}{2} \cdot (y_j - o_j)^2$; total $E = \sum E_j$.

Con los números: $E_1 \approx 0.274811, E_2 \approx 0.02356; E \approx 0.29837$.

Gradientes (capa de salida, peso $w_5: h_1 \rightarrow o_1$): $\partial E / \partial w_5 = (o_1 - y_1) \cdot \sigma'(a_{o_1}) \cdot h_1$; actualización: $w_5 \leftarrow w_5 - \eta \cdot \partial E / \partial w_5$, con $\eta=0.5$ en el ejemplo.

Forma vectorizada con deltas: $\delta^{\{o\}} = (o - y) \odot \sigma'(a_o)$; $\delta^{\{h\}} = (W^2)^T \cdot \delta^{\{o\}} \odot \sigma'(a_h)$;
 $\partial E / \partial W^2 = \delta^{\{o\}} \cdot h^T$; $\partial E / \partial b^2 = \delta^{\{o\}}$; $\partial E / \partial W^1 = \delta^{\{h\}} \cdot x^T$; $\partial E / \partial b^1 = \delta^{\{h\}}$.

2.1 Pérdidas de uso común (completando la teoría)

- BCE (binaria): $L = -[y \cdot \log(o) + (1-y) \cdot \log(1-o)]$.
- CCE (multiclase + softmax): $\text{softmax}(z)_k = e^{\{z_k\}} / \sum_j e^{\{z_j\}}$; $L = -\sum_k y_k \cdot \log(\text{softmax}(z)_k)$.

3) Implementación: activaciones, regularización y optimización

Salidas según tarea:

- Regresión: activación lineal (o ReLU/sigmoide si se requiere rango).

- Binaria: sigmoide + BCE.
- Multiclase excluyente: softmax + CCE.
- Multietiqueta: sigmoide por neurona + BCE.

Regularización:

- L2: añade $\lambda \cdot ||W||^2$.
- L1: añade $\lambda \cdot ||W||_1$.
- Dropout (p): apaga activaciones aleatorias en train.
- Early stopping: detiene por métrica de validación.
- Data augmentation: transformaciones verosímiles (especialmente útil en imágenes).

3.1 Optimizadores (con fórmulas)

SGD: $\theta_{t+1} = \theta_t - \eta \cdot \nabla L(\theta_t)$. Keras: SGD(lr=...).

Momentum: $v_{t+1} = \mu \cdot v_t - \eta \cdot \nabla L(\theta_t)$; $\theta_{t+1} = \theta_t + v_{t+1}$. Keras: SGD(momentum=0.9).

Nesterov: gradiente en $\theta_t + \mu \cdot v_t$. Keras: SGD(momentum=0.9, nesterov=True).

AdaGrad: $s_{t+1} = s_t + g_t \odot g_t$; $\theta_{t+1} = \theta_t - \eta \cdot g_t \oslash (\sqrt{s_{t+1}} + \epsilon)$.

RMSProp: $s_{t+1} = \rho \cdot s_t + (1 - \rho) \cdot g_t \odot g_t$; $\theta_{t+1} = \theta_t - \eta \cdot g_t \oslash (\sqrt{s_{t+1}} + \epsilon)$.

Adam: $m_{t+1} = \beta_1 \cdot m_t + (1 - \beta_1) \cdot g_t$; $v_{t+1} = \beta_2 \cdot v_t + (1 - \beta_2) \cdot g_t \odot g_t$;

$\hat{m} = m_{t+1} / (1 - \beta_1^{t+1})$; $\hat{v} = v_{t+1} / (1 - \beta_2^{t+1})$; $\theta_{t+1} = \theta_t - \eta \cdot \hat{m} \oslash (\sqrt{\hat{v}} + \epsilon)$.

3.2 Guía rápida de arquitectura

- Capas: una sola oculta puede aproximar funciones complejas; más capas suelen mejorar representación.
- Ancho: pirámide decreciente o ancho constante (ambas son prácticas comunes).
- Hiperparámetros: LR ($1e-1 \dots 1e-4$), épocas por validación, λ de regularización, p de dropout, tamaño de batch.

4) Resumen

El perceptrón aprende hiperplanos y sirve para tareas lineales; las MLP con activaciones no lineales más backprop resuelven problemas complejos. Para clasificar, usar activaciones y pérdidas acordes (sigmoide/softmax + BCE/CCE). Para generalizar, aplicar regularización, early stopping y augmentation. Para entrenar eficientemente, preferir Adam/RMSProp o SGD con momentum/Nesterov.

Ejercicio 3

Explique brevemente en qué consiste y para qué se utiliza el método **Early Stopping** en Redes Neuronales.

En qué consiste.

Early Stopping es una técnica de regularización “por tiempo de entrenamiento”: durante el aprendizaje por backpropagation se separa un conjunto de validación y se monitorea su pérdida (o accuracy) al final de cada época. Si la métrica de validación deja de mejorar y, tras un margen mínimo (Δ) y un número de épocas de tolerancia (patience), empeora, se detiene el entrenamiento y se conservan los pesos del modelo en la época donde la validación fue mínima.

Para qué se utiliza.

Su objetivo es evitar el sobreajuste (overfitting) y, a la vez, reducir el tiempo de entrenamiento. En redes neuronales, la pérdida de entrenamiento tiende a disminuir de forma monótona, mientras que la de validación típicamente baja al principio y luego sube cuando el modelo comienza a memorizar el conjunto de entrenamiento. Early Stopping corta en el “valle” de la curva de validación, aproximando el mínimo del error de generalización. Puede verse como una regularización equivalente a limitar la “capacidad efectiva” del modelo mediante menos actualizaciones de gradiente.

Implementación típica (resumen):

1. Partir el dataset en entrenamiento/validación.
2. Entrenar por épocas registrando la pérdida de validación.
3. Si no hay mejora mayor que min_delta durante patience épocas, stop.
4. Restaurar los pesos de la mejor época (best checkpoint).

Ejercicio 6

Indique si las siguientes afirmaciones sobre **Redes Neuronales** son verdaderas o falsas (V/F):

- a) El perceptrón simple no permite dividir el espacio linealmente: _____
- b) Las redes SOM necesitan un conjunto de datos balanceado y correctamente etiquetado para su entrenamiento: _____
- c) Para poder entrenar una red neuronal con el algoritmo **Backpropagation**, las funciones de activación de las neuronas deben si o si ser derivables: _____
- d) Backpropagation es un método alternativo al método de descenso por gradiente: _____

a) Falso. El perceptrón simple sí divide linealmente: implementa un hiperplano $w^T x + b = 0$. Lo que no puede es separar problemas no linealmente separables (p. ej., XOR).

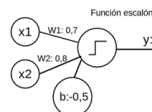
b) Falso. Las SOM (Self-Organizing Maps) se entrenan no supervisadas: no requieren etiquetas ni un dataset balanceado; aprenden la topología de los datos.

c) Verdadero. Backpropagation necesita funciones de activación (al menos) derivables para propagar gradientes. En la práctica alcanza con derivabilidad por tramos (p. ej., ReLU usa subgradiente en 0).

d) Falso. Backpropagation no es alternativo a descenso por gradiente: es el método para calcular los gradientes de forma eficiente; luego esos gradientes se usan con GD/SGD/Adam, etc.

Ejercicio 7

Dado el siguiente perceptrón, calcule los valores de salida en cada uno de los casos:



$$u(x) = \begin{cases} 0 & \text{si } x < 0 \\ 1 & \text{si } x \geq 0 \end{cases}$$

- a. $x_1 = 0, x_2 = 0$: _____
- b. $x_1 = 1, x_2 = 1$: _____
- c. $x_1 = 0.5, x_2 = 0.5$: _____
- d. $x_1 = -0.25, x_2 = 1$: _____

Regla: $y = u(a)$ con $a = w_1 x_1 + w_2 x_2 + b$.

Parámetros: $w_1 = 0,7$, $w_2 = 0,8$, $b = -0,5$.

$$u(x) = \begin{cases} 0 & x < 0 \\ 1 & x \geq 0 \end{cases}$$

a) $x_1 = 0, x_2 = 0$:

$$a = 0,7(0) + 0,8(0) - 0,5 = -0,5 < 0 \Rightarrow y = 0.$$

Explicación: la suma ponderada no alcanza el umbral (negativa).

b) $x_1 = 1, x_2 = 1$:

$$a = 0,7 + 0,8 - 0,5 = 1,0 \geq 0 \Rightarrow y = 1.$$

Explicación: la activación es positiva: cruza el umbral.

c) $x_1 = 0,5, x_2 = 0,5$:

$$a = 0,7(0,5) + 0,8(0,5) - 0,5 = 0,35 + 0,4 - 0,5 = 0,25 \geq 0 \Rightarrow y = 1.$$

Explicación: la contribución conjunta supera al sesgo negativo.

d) $x_1 = -0,25, x_2 = 1$:

$$a = 0,7(-0,25) + 0,8(1) - 0,5 = -0,175 + 0,8 - 0,5 = 0,125 \geq 0 \Rightarrow y = 1.$$

Explicación: aunque x_1 resta, x_2 compensa y la activación queda no negativa.

Ejercicio 8

Indique si las siguientes afirmaciones sobre implementación de **Redes Neuronales** son verdaderas o falsas (V/F):

- a) Dropout es un método de regularización que evita el sobre-ajuste: _____
- b) Habitualmente Adam suele ser más rápido que RMSProp y que Momentum como optimizador: _____
- c) La regularización L2, L1 nos evita tener que entrenar la red neuronal con Backpropagation : _____
- d) La función Softmax permite ponderar la salida como un puntaje o probabilidad de la certeza en la clasificación realizada: _____

a) Verdadero. Dropout apaga aleatoriamente neuronas en entrenamiento, reduce co-adaptaciones y actúa como regularización, mitigando el sobreajuste.

b) Verdadero (en general). Adam usa tasas adaptativas + momentum (m y v), y suele converger más rápido que RMSProp o SGD con momentum en la práctica inicial; no siempre es el mejor final.

c) Falso. L1/L2 agregan penalizaciones a la función de pérdida, pero igual se entrena con backpropagation para calcular gradientes (incluyendo los términos de regularización).

d) Verdadero. Softmax transforma logits en probabilidades normalizadas por clase ($\sum p_i = 1$), interpretables como grado de certeza de la clasificación (aunque su calibración puede variar).

Ejercicio 5

Determinar si las siguientes afirmaciones son verdaderas o falsas:

- Las redes neuronales se optimizan mediante descenso de gradiente. **V** _____
- Las redes neuronales son sensibles a la escala de los datos. **V** _____
- Un perceptrón simple no puede modelar compuertas NAND. **F** _____
- Las redes neuronales muy complejas pueden sub-ajustar, esto se soluciona con métodos de regularización como Early stopping. **F** _____

V

El entrenamiento minimiza una función de pérdida usando descenso por gradiente (SGD/Adam, etc.). Backpropagation solo calcula los gradientes; la actualización es por GD.

V

Las RN son sensibles a la escala: la magnitud de las entradas afecta los gradientes y la convergencia (p. ej., activaciones saturan en sigmoide/tanh). Por eso se normaliza/estandariza y se usan capas como BatchNorm.

F

Un perceptrón simple sí puede modelar NAND porque es linealmente separable (igual que AND/OR). Lo que no puede son funciones no linealmente separables (p. ej., XOR).

F

Las redes muy complejas tienden a sobre-ajustar, no a sub-ajustar. Y técnicas de regularización como early stopping están diseñadas para reducir el sobreajuste, no para resolver el subajuste.